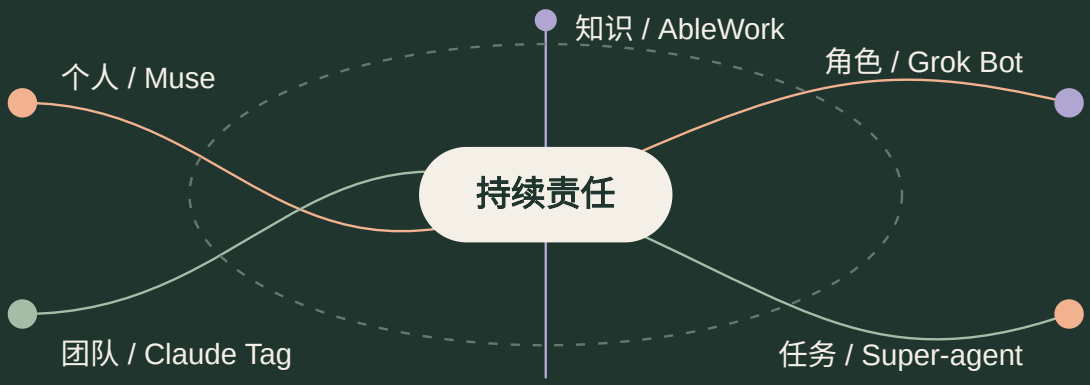
AGENT EVOLUTION · 研究专刊

智能体的演进

从个人代理、数字角色与团队协作，
到持续任务和可复用的研究知识。



> 当智能体开始持续工作，差异不只在于能做多少事，
> 还在于责任怎样接续、边界怎样保持、成果怎样留下。

作者 / Hanlin Li
资料截至 / 2026.11.01　编成 / 2026.11.02

目录 1 / 2

左侧为正文节号，右侧为 PDF 页码；电子版可点击标题跳转。

01	从一次回答，到一份持续责任	4
02	这份材料写给谁，应该怎样使用	5
03	先把容易混淆的词说清楚	6
04	五种出发点，五种工作中心	7
05	证据分层：产品承诺、运行事实与作者推断	8
06	Muse：从一个人的生活连续性出发	9
07	Muse 的委托链：目标、计划、行动、回看	10
08	Muse 的信任基础：记住偏好，也允许纠正	11
09	Muse 的权限：执行能力之外需要独立裁定	12
10	Muse 的目标与代价：减少生活管理负担	13
11	Grok Bot：把反复的工作交给长期角色	14
12	共享电脑：协作方便，也改变边界假设	15
13	从一次操作，到 Skill 与 Routine	16
14	多 Bot 协作：谁交接，谁承担完成责任	17
15	Grok Bot 的目标与代价：可委托的重复工作	18
16	Claude Tag：工作已经发生在团队讨论里	19
17	原线程交付，让异步工作仍然可理解	20
18	团队记忆：共享上下文仍需要范围	21
19	Agent 身份：谁的工具，谁的行动	22
20	Claude Tag 的目标与代价：团队少搬运一次上下文	23
21	Super-agent：从事件不断到来的工程工作出发	24
22	从事件到任务：看见变化，不等于可以行动	25
23	等待、失败与恢复：让人知道为什么停在这里	26
24	低打扰与执行边界：原型的启发也有适用范围	27
25	Super-agent 的目标与代价：减少人维持任务的工作	28
26	横向比较一：谁是工作的中心	29
27	横向比较二：触发、准备、通知、行动	30
28	横向比较三：持续的是任务，还是机器	31
29	横向比较四：记忆、流程与知识分别更新	32
30	横向比较五：目录、工单与向量检索可以共存	33

目录 2 / 2

左侧为正文节号，右侧为 PDF 页码；电子版可点击标题跳转。

31	横向比较六：权限沿着什么传递	34
32	横向比较七：结果究竟留在哪里	35
33	横向比较八：成本与注意力要一起计算	36
34	横向比较九：质量应该在哪些地方被检查	37
35	五种形态怎样组合，而不让责任丢失	38
36	AbleWork：研究工作的知识托管与边界治理	39
37	主路径：从研究成果，到后续问题的输入	40
38	运行规模：说明有人使用，也保留统计边界	41
39	脱敏案例：一人的新版研究进入另一人的问题	42
40	历史完整编译：实际完成了什么	43
41	为什么日常采用梗概，完整编译另行选择	44
42	AutoWiki 与 AbleWork：语义编译和宿主责任	45
43	主动准备：明确关注以后，后台继续检查	46
44	只读公共面与候选池：分享不等于直接改写	47
45	报告、社区知识与 commons 不是同一块公共层	48
46	失败与反馈：把运行问题变成产品改动	49
47	当前完整度：已运行、历史验证、待补证据	50
48	AbleWork 的结合路线：先增强已有工作	51
49	情境推演：一份持续研究委托怎样分工	52
50	情境推演：多人接力修复一个问题	53
51	情境推演：来源更新或撤回之后怎么办	54
52	情境推演：个人发现怎样成为公共知识	55
53	未来判断一：入口会更自然，责任对象会更明确	56
54	未来判断二：Agent 身份会成为应用设计的一部分	57
55	未来判断三：可携带成果比无限对话更耐久	58
56	未来判断四：多个形态会并存，运营能力成为分水岭	59
57	怎样做一轮公平、可复核的比较	60
58	建设路线：从一个可靠案例，走向持续职责	61
59	公开来源：外部产品与研究原型	62
60	AbleWork 依据、统计口径与研究限制	63

01

从一次回答，到一份持续责任

智能体开始承担持续工作以后，产品的难题发生了变化。一次回答是否正确仍然重要，但用户还会追问：我离开后事情会不会继续？明天回来，它是否记得已经做过什么？同事接手时，能否看见依据？曾经允许读取的材料，现在还能不能使用？这些问题决定了智能体能够承担多大的责任。

本报告比较 Muse、Grok Bot、Claude Tag、DeepSeek Super-agent 和 AbleWork。五者分别以个人目标、角色职责、团队讨论、持续任务和研究对象为主要组织中心。这是对产品出发点的归纳，不是排他分类。一种产品完全可能同时拥有其他几种能力。比较的目的，是找到不同选择解决的问题，以及它们把什么成本留给了用户和系统。

核心判断是：后续智能体的差异，将越来越多地体现在责任如何持续、权限如何约束、结果如何留下。 模型能力提供行动的可能；任务记录、身份、工具、交付和知识更新共同决定行动能否进入日常工作。入口越自然，背后的责任划分反而越需要明确。

AbleWork 在这组比较中的价值，是提供一个研究协作场景。研究员和 Agent 产出报告，平台保留来源、版本与读权，再让后来的问题使用这些成果。它还提供就地委托和有边界的后台准备，因此不能只被描述为“个人 Agent 干完活以后的存储层”。知识托管与边界治理是其主要定位，研究工作的组织是实现这个定位所需的产品部分。

对建设者而言，优先顺序应从具体责任出发：先选一类反复发生、能够验收的工作，再确定工作如何接续、在哪些地方停下来、结果由谁采用。过早追求角色数量、主动消息数量或知识页数量，容易让系统显得繁忙，却无法说明用户少承担了什么。

本报告没有对五个产品做同条件性能测试。官方说明用于确认定位与机制；AbleWork 的运行汇总用于说明实际采用；跨产品的比较和未来判断属于作者分析。全文截至 2026 年 11 月 1 日，封面月份为 2026 年 11 月。

02

这份材料写给谁，应该怎样使用

主要读者是正在决定 Agent 产品形态的产品负责人、技术负责人和架构师。报告希望帮助他们回答一项具体决策：**我们准备把哪一种持续责任交给系统，为此需要建设什么，而哪些事情可以暂时不做？** 对研究团队负责人，另一项问题同样重要：一人的成果怎样减少另一人重新找材料、核对版本和解释背景的工作？

如果正在讨论定位，可先读五个产品的出发点，再读横向比较。讨论“我们也要做主动 Agent”时，应把主动拆开：后台检查变化、准备草稿、提出建议、公开发言、修改业务系统，分别需要不同授权。用一个形容词概括它们，会掩盖产品真正要做的选择。

如果负责工程落地，可从任务状态、记忆、权限和结果交付几节进入。关键不是照搬完整架构，而是找到当前最常见的失败。例如用户反复解释旧背景，首先需要可靠的接续记录；任务中断后重复发消息，需要的是外部操作回执；资料过期仍进入回答，则需要版本与失效处理。不同故障对应不同工程动作。

如果负责 AbleWork 或类似知识应用，重点看后半部的三条路径：日常梗概与检索、历史完整知识编译、个人成果进入公共候选。它们具有不同目标和证据强度。把三条路径合写成一个“已经完成的知识循环”，会让读者误判当前应用；把它们全部写成未来规划，又会抹掉已经存在的用户价值。

阅读任务	建议阅读范围	带走的判断
确定产品方向	01—05、26—35	工作中心与责任边界
深读具体形态	06—25	出发点、机制、代价
评估研究平台	36—48	运行证据与实际缺口
制定验证方案	49—58	场景、指标与建设顺序
核对依据	59—60	来源、日期与口径

报告中的流程图都属于机制示意，具体实施以相应产品资料为准。案例标记为“情境推演”时，表示用一个假设任务检查设计，不能当作产品实测记录。读者可以把情境中的对象换成自己的工作，保留授权、等待和验收问题，形成内部讨论材料。

03

先把容易混淆的词说清楚

“智能体”在本文指能够围绕目标选择行动、调用工具并根据结果继续工作的系统。聊天界面只是入口之一。一个系统是否持续工作，也不能只看进程是否常驻：真正需要持续的是目标、约束、任务状态和结果之间的关系。模型可以按事件唤醒，工作记录仍然连续。

任务有相对明确的完成条件，例如准备一份比较表；**职责**反复产生任务，例如每周核对某类变化；**目标**允许路径调整，例如持续改善某项研究质量。三者不能混作同一种委托。目标越开放，系统越需要解释为什么选择下一步，以及什么变化会使旧授权不再适用。

“记忆”也至少有几种对象。偏好记忆帮助减少重复沟通；任务记录帮助接续执行；流程知识说明如何做事；领域知识保存事实、判断及其依据。一个用户喜欢简短回答，不等于某项业务判断已经得到验证。把这些内容放进同一份长提示词，可能方便初次实现，却会让更新和纠错变得含糊。

“主动”涉及发现变化、判断意义、准备结果和打断用户几个步骤。后台发现新材料，并不必然需要立即通知。系统可以先准备一张有出处的卡片，等用户回到相关对象时呈现。相反，一项即将失效的审批可能值得立即提醒。评价主动性，应看被处理的时机是否合适。

“边界治理”在本文有具体含义：确认谁发起工作、允许读取什么、允许修改什么、结果对谁可见，以及权限或来源改变后如何处理。它不只是提示词里的一句“注意安全”。“知识编译”则指把源材料与已有知识比较，形成有结构、可更新、可追溯的内容；简单摘要可以是其中一种输入，不能自动代表全部过程。

这些区分使比较更公平。一套工单系统可能擅长状态接续，却没有正式知识更新机制；一个知识平台可能擅长跨作者复用，却并不适合控制个人账户里的所有应用。评价应该跟着任务走，避免把不同工作中心的产品放在同一条能力分数线上。

04

五种出发点，五种工作中心

产品形态往往由最初想解除的用户负担决定。个人事务散在多个应用中，容易产生围绕“我”的持续代理；同一类操作反复发生，容易产生长期角色和例行流程；团队已经在讨论工作，适合把 Agent 放进原讨论；工程事件连续到来，需要明确任务与执行者；研究成果跨人、跨时间使用，则需要稳定的知识对象。

FIG. 01 / 机制示意

五种工作中心



五条路径并列存在；工作中心影响委托、状态和交付，不表示能力排名。

这张图没有高低顺序。一个人的旅行安排和团队的代码修复，不应共享同一套完成标准。旅行任务可能在预订确认后结束；代码任务可能在审查、合并和部署后才完成；研究结论甚至没有永久终点，后续来源变化会重新打开问题。工作中心决定什么信息需要长久保留。

入口也会影响用户预期。给一个有名字的 Bot 发消息，容易让人期待长期理解与主动安排；在频道里 @Claude，容易期待同事能够看到过程；在报告旁委托 Able，则自然期待结果仍与那份报告、那个主题相关。界面看起来只是按钮和聊天框，其实已经向用户承诺了一种责任关系。

本报告将这种关系拆成同一套问题：谁委托，围绕什么对象，怎样开始，工作在哪里运行，等待什么事件，结果交给谁，怎样在下一次工作中继续使用。比较不要求每个产品都给出同一种答案，但要求答案前后一致。

对 AbleWork 最有帮助的学习方式，是借用其他形态中适合研究工作的部分：个人代理的低沟通负担，角色流程的可重复方法，频道 Agent 的原处交付，持续任务系统的等待与恢复。组合时仍需保留研究对象的来源、版本和共享范围，不能让入口体验覆盖这些约束。

05

证据分层：产品承诺、运行事实与作者推断

一份比较报告最容易出现的问题，是把不同性质的证据写成同一种事实。官方说支持某项能力，证明的是产品公开承诺或文档描述；一段运行记录证明某次路径确实发生；长期稳定性、任务质量和节省时间，还需要另外的数据。本报告按这几个层次组织材料。

标记	表示什么	不能据此推出什么
官方说明	厂商或项目维护者公开描述	作者已亲自测试全部功能
运行汇总	作者对自有项目记录的核查	独立第三方审计、普遍效果
作者分析	根据机制推导收益与代价	产品已提供相应新功能
情境推演	用假设案例检验设计	真实客户案例或性能结果
建议	后续可实施和验证的方案	已排期、已上线或已获采用

外部产品以官方介绍、官方使用文档及公开研究代码为主。本文不使用不透明排行榜，也不把某家内部开发比例换算成普遍生产率。不同文档可能处于不同更新时点；涉及关键差异时，采用截至报告日期可读取的具体页面，并在来源页列出地址。

AbleWork 部分使用公开项目介绍与作者自有核查汇总。运行材料保留两个时间口径：近期使用窗口，以及较早的一次完整编译批次。两者分别证明日常采用和历史可执行性，不能相加形成新的“月度成果”。内部用户身份、业务材料和服务地址不公开，读者据此应把相关数字视为作者披露。

未来形态一章采用条件判断。例如“如果角色交接能够携带约束和回执，多角色协作才可能减少人工协调”。这种表述允许被后续结果推翻，比断言某种形态一定取代其他形态更有用。报告也列出反例：当任务短、权限简单、结果无需复用时，普通助手可能已经足够。

这份材料的贡献主要是分析框架和工程取舍。它不提供五个产品在同一任务上的准确率、成本或安全优劣排名。要做采购或迁移决定，仍应使用本报告后半部的验证方法，在自己的任务、资料和权限环境中形成可比较证据。

06

Muse：从一个人的生活连续性出发

Muse 的公开定位是个人 AI Agent。它通过应用或 WhatsApp 接收委托，在个人专属的云端环境中使用工具，继续推进任务与长期目标。手机承载日常沟通，实际执行并不等于全部发生在手机本地。用户可以决定连接哪些服务，以及授予多少访问权限。 [M1]

这个出发点值得单独讨论。个人工作和生活并不天然按软件划分：一封邮件可能影响日程，日程又影响出行和采购。用户记住的是“把这件事安排好”，软件却要求逐一操作。围绕个人建立持续代理，可以尝试保存任务之间的关联，让用户少做一次又一次的背景转述。

个人目标也带来特殊困难。目标经常没有清晰的完成线，偏好甚至相互冲突。某人想减少出行费用，同时不愿乘坐过早航班；想规律训练，却需要照顾临时家庭安排。系统若只优化一个容易计算的指标，可能完成了表面任务，却偏离真实意图。因此持续代理需要保留约束的来历和优先顺序。

作者分析：Muse 最有代表性的变化，是把“理解这个人”作为跨任务的连续关系。 这种关系可以降低沟通成本，但会使用户对记忆的纠正、对行为的解释提出更高要求。一个记错的偏好，若只影响一次回复，代价有限；若持续影响多个后台任务，就会反复制造偏差。

适合这种形态的任务通常跨应用、反复发生，又有个人特征。它们的价值并不一定表现为复杂推理，可能只是把几件容易遗漏的事情按时完成。评价时应记录用户还需要补充多少背景、是否经常撤销安排、哪些提醒真正促成了行动，而不能只统计 Agent 做了多少步骤。

对 AbleWork 的启发，是长期关注也应围绕研究员的真实问题组织。用户关注某个主题，系统应保存关注原因与有效期限，而不只是保存关键词。这样后续材料到来时，才可能解释它与既有研究的关系。这个启发属于设计借鉴，不表示两者当前拥有相同的个人代理范围。

07

Muse 的委托链：目标、计划、行动、回看

Muse 的设计说明把持续对话、侧边话题、目标视图和活动记录放在同一体验中，并描述了后台工作完成后判断是否值得通知的做法。它还允许产出文档、页面等可独立使用的成果。这些是官方产品设计描述，本文没有进行完整体验测试。[M2]

FIG. 02 / 机制示意

个人目标如何继续推进



机制示意 · 箭头表示工作关系，不表示耗时或性能

用户的目标与约束贯穿过程；涉及重要外部后果时另行检查权限。依据 M1、M2 简化。

从产品分析看，这条链中最容易被忽略的是回看。用户中途回来，关心的往往不是执行过哪些工具，而是当前还有哪些未决事项。好的接续摘要应能够回答：目标有没有变化，哪些部分已经完成，哪些依赖外部回应，接下来需要用户决定什么。工具日志可以作为依据，不能代替面向人的状态说明。

设想用户安排一场家庭聚会。邀请名单、饮食限制和日期属于约束；比较场地属于准备；发出邀请和支付订金属于外部行动。把它们写成一个长任务，并不会消除边界。相反，系统需要明确哪些准备工作可以继续，哪些行动在条件变化后应重新检查。此例是情境推演，不是 Muse 实测案例。

任务之间还可能争夺资源。两个目标同时占用同一天下午，即使分别规划合理，也不能同时执行。持续代理需要一种能够暴露冲突的对象视图，而非只靠消息先后顺序。由此可以推导，未来的个人 Agent 界面会需要比聊天记录更稳定的计划、承诺和待决事项表达。

这也解释了为什么“像与人聊天”只是入口优势。使用者可以自然表达，系统仍须把自然语言转成可检查的工作对象。个人代理的成熟程度，最终体现在对含糊目标和变化约束的处理，而不是聊天气泡是否像真人。

08

Muse 的信任基础：记住偏好，也允许纠正

个人代理越有用，往往越需要持续接触敏感而零散的背景。用户今天提到的一次临时偏好，未来可能被理解为长期规则。产品因此需要让记忆能够被检查和修正。Muse 的公开说明包含可查看和编辑的记忆文件，以及让它忘记特定信息的能力。 [M1] [M2]

作者分析：记忆的产品价值来自少解释，记忆的产品成本来自错解释。 两者应一起衡量。假如系统记得用户偏好火车，后续建议更省心；如果某次出差必须乘飞机，旧偏好就不应阻止完成新任务。事实、偏好、临时要求、推测之间最好存在可辨认的差别。

可以用四个问题检查个人记忆：这条内容由谁提出，适用范围是什么，最后确认在何时，新的指令是否覆盖了它。无须把所有记忆都变成复杂数据库，但影响重大行为的记忆，至少应该保留这些语义。否则模型每次都在长历史中重新猜测，长期一致性很难解释。

记忆纠正还需要处理下游影响。用户撤销一个偏好以后，已生成但尚未执行的计划是否更新？旧文件是否继续误导另一个任务？系统不能简单回答“我忘了”，却让过时内容留在后台任务里。删除一段文本、撤销一个授权、改变一项计划，是不同操作。

在研究工作中，这对应着“研究偏好”和“领域判断”的分离。某研究员喜欢先看行业供需，不意味着供需就是某家公司唯一重要的因素。平台可以用偏好安排展示顺序，但不应把偏好当作事实权重。AbleWork 若学习个人代理的记忆体验，应优先学习可纠正性，而非无条件扩大记忆范围。

实际验证可以从一次可控纠正开始：先给系统一个偏好，运行若干相关任务，再明确修改它，观察新任务、旧计划和展示界面是否一致变化。这比询问“你还记得什么”更接近真实可靠性，也能暴露记忆与任务状态之间是否真正相连。

09

Muse 的权限：执行能力之外需要独立裁定

Meta 的安全说明描述了运行环境与安全服务的隔离，以及由 Sentinel 对连接器行动和网络出口作权限判断。凭据服务与主执行环境分开；敏感行动可转成人的审批。这里引用的是其公开架构说明，不能据此断言系统不存在漏洞。 [M3]

值得借鉴的是职责分离。提出行动的模型可能正在读取网页、邮件和文件，而这些输入未必可信。若同一个上下文又能修改自己的权限规则，外来内容就可能影响授权判断。把权限裁定放在执行者不能随意改写的位置，可以减少“看见一段指令就把它当成用户要求”的风险。

审批本身也需要具体。用户批准“处理旅行”与批准“向某商户支付某金额”不是同一件事。一次审批应绑定对象、操作、用途和范围。否则用户以为只批准眼前一步，系统却把同一句话解释成长期权限；或者用户已经明确授权，系统仍为每次低影响操作重复打断。

个人场景还需要避免审批疲劳。凡事确认，会把原本希望节省的注意力重新消耗在弹窗上。较合理的路径是让只读准备按既定范围继续，把审批集中在费用、对外承诺、删除、权限变化等后果上。具体阈值应由任务和用户控制，不能由一个抽象的“积极程度”替代。

报告特意保留一个时间边界：Muse 的相关发布材料把 Confidential VM 描述为后续计划。现有专属环境、安全隔离与“服务提供者在密码学意义上无法访问”是不同保证。本文不会把计划中的保证写成已经普遍交付。 [M3]

对 AbleWork 而言，对应问题不是复制个人电脑安全架构，而是让材料权限、正式知识写入和公共发布不由读取材料的 Agent 自行决定。机制可以不同，职责关系应相似：执行者准备结果，宿主依据真实身份与当前政策判断结果能去哪里。

10

Muse 的目标与代价：减少生活管理负担

Muse 形态的目标可以概括为减少一个人协调零散事务的负担。其成败应放在长期关系中观察：用户是否更少遗漏事项，是否更少反复解释，是否愿意持续授权，以及错误发生后能否迅速恢复信任。单次任务演示很难回答这些问题。

这种形态的主要代价是个性化积累与授权复杂度相互增长。接入服务越多，可安排的事情越丰富，但权限也更难让普通用户理解。系统需要区分“它知道什么”和“它能做什么”，还要让用户看见某个结果使用了哪些来源。一个温和、自然的角色形象不能代替这些说明。

另一个代价是依赖性。用户把很多约束交给代理之后，迁移或暂停服务会变得困难。可导出的计划、偏好和待办记录，具有实际产品价值；它们使用户能够保有工作连续性。对于长期 Agent，退出体验应与接入体验一同设计，这也是检验其是否真正以用户为中心的方法。

作者判断：个人持续代理可能成为跨应用事务的重要入口，但不会自动覆盖组织知识治理。 多人共享材料时，需要处理署名、争议、版本、组织权限和正式采用。个人偏好越深，反而越需要与公共知识分离。一个人为方便自己形成的摘要，不应未经检查就代表团队共识。

应检验的收益	对应观察	需要防止的误读
少解释背景	同类任务的补充说明次数	对话变短就一定更准确
少遗漏事项	用户确认的遗漏减少	提醒更多就更有用
安排更合适	修改和撤销原因	执行动作多就更成功
信任可恢复	错误纠正后的后续表现	一次道歉等于系统修复

AbleWork 可以从这里获得一个明确方向：长期关注要服务于人的研究目标，并让用户能够修改、暂停和回看。它无需先成为覆盖私人生活的通用代理，才能做好持续研究支持。把适用范围限定清楚，反而有助于形成可验证的产品承诺。

11

Grok Bot：把反复的工作交给长期角色

Grok Bot 的官方文档把 Bot 描述为有名字、职责和持续上下文的协作者。它们可在持久云端电脑上使用浏览器、文件和终端，处理跨应用任务。多个 Bot 可以协作并移交工作。这里讨论的是这一执行产品，而不是把 X 上的公开问答账号直接当作完整工作系统。 [G1]

角色是这种形态的组织中心。用户不一定每次从零写任务，而是先设定某个角色负责什么，再向它委托相关工作。这接近现实团队中的分工：同一个研究支持角色能够持续理解输出习惯，同一个运营角色能够重复处理相似流程。用户希望减少的，是每次重新配置工具与上下文的负担。

作者分析：角色的价值在于稳定职责，而不在于增加拟人名称。 如果两个 Bot 读取同样的材料、使用同样的工具、接受同样的任务，分别命名并不会自动提高质量。只有在职责、输入、输出和验收方式确有差异时，角色划分才有清晰收益。

角色也会形成新的沟通成本。用户究竟应该找谁？一个角色发现另一角色的结果有误，谁负责修正？如果用户仍需在多个对话间复制信息，分工就没有实现其原本目标。因此多角色产品必须提供工作归属和交接记录，避免让用户成为所有 Agent 之间的人工路由器。

Grok Bot 对后续形态的启发，是把一次成功任务转成可重复职责。产品不必一开始就建立复杂自动化图，可以先让用户委托一个具体工作，观察稳定方法，再逐渐形成例行流程。不过，对角色的信任应该来自完成记录，而非头像、名字或系统对自己的能力描述。

放到 AbleWork 场景中，适合借鉴的是研究方法的可复用表达。资料筛选、版本比较、引用核查可以成为相对稳定的职责；最终判断和正式发布仍需保留明确责任人。角色可以协助研究工作，但不会因为被命名为“审稿人”，就自动获得对公共知识的裁定权。

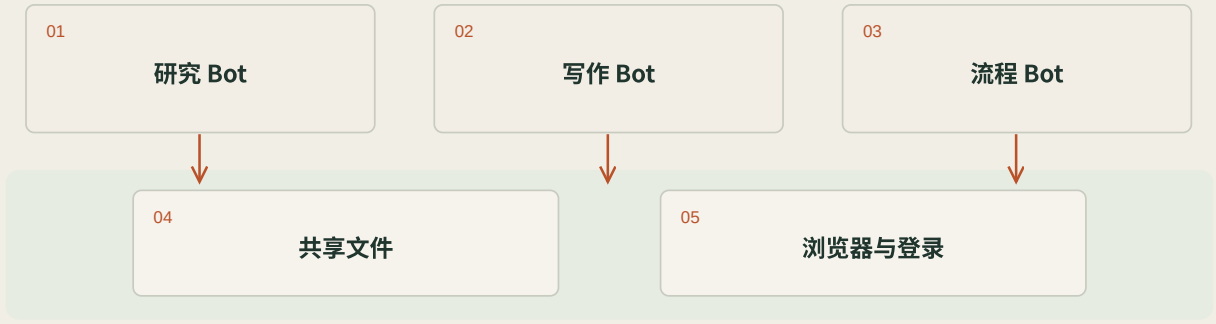
12

共享电脑：协作方便，也改变边界假设

Grok Bot 文档明确说明，同一用户的 Bots 共享一台云端电脑上的文件、浏览器会话和应用登录。各 Bot 的对话和学习上下文通常分开，部分个人信息共享。这意味着“有多个角色”与“存在多个隔离的权限主体”不能画等号。[G1]

FIG. 03 / 机制示意

角色分开，账户内环境共享



上方是不同角色，下方是同一用户共享环境；角色记忆与环境访问是不同关系。依据 G1。

共享环境的直接收益，是交接不必从头准备。一个 Bot 产生的文件，另一个可以继续加工；已完成的应用连接，也不必为每个角色重复建立。在同一用户的工作范围内，这样的便利可能比每个任务完全隔离更符合使用习惯。

代价是角色边界不能只靠文件夹名称或提示词。假设用户让一个 Bot 处理客户材料，另一个负责公开内容，两个角色共享登录和文件时，后者是否可读取前者的数据，需要由真实访问与行动控制决定。仅仅告诉它“你是公开内容助理”，不构成可以依赖的隔离保证。此例属于边界分析，不是对产品漏洞的断言。

协作还会遇到共享状态竞争。同一个文档被同时修改，同一个浏览器登录状态发生变化，或两个任务都以旧数据生成结果，都可能导致工作不一致。多角色并行因此需要明确哪些资源可以同时读、哪些写入需要串行，以及失败后由谁恢复。并行数量本身不是收益证明。

对 AbleWork 的对应启发，是严格区分三件事：多个 Agent 的分工、不同用户的访问权、正式知识的发布权。前者可以灵活编排，后两者应由宿主系统依据身份执行。合作不要求取消所有边界；可靠的交接应携带结果和必要上下文，而非默认移交所有可访问资源。

13

从一次操作，到 Skill 与 Routine

Grok Bot 的官方说明区分 Skill 与 Routine：前者保存如何做一项工作，后者规定何时由某个 Bot 执行。文档还描述了通过示范产生流程草稿，并要求补充验证方法、失败处理与审批边界。示范功能可能逐步开放，本文不假定所有用户都已拥有。 [G2]

这个区分有实用价值。一项流程可能方法正确，却在错误时间运行；也可能按时启动，但读取了过期输入。把方法与触发分开，可以分别修正。用户改变每周执行日期，不应意外改变研究方法；更新筛选规则，也不应悄悄新增对外发送行为。

作者分析：可重复流程的核心是条件，而非点击顺序。 浏览器演示能记录走过的路径，但不能从一个成功样本中得出全部例外。来源缺失、登录过期、结果为空、版本冲突和权限变化，常常没有出现在示范里。若这些情况没有写清，自动化只是把原本由人临场判断的工作隐藏起来。

考虑一个每周客户信息整理任务。正常路径是读取名单、检查变化、形成草稿。若名单为空，系统应区分“本周确实没有对象”与“读取失败”；若同一客户出现两次，应保留去重依据；若准备发送结果，应单独确认收件人与共享范围。这些条件比保存一串工具调用更接近可用方法。

在研究平台中，Skill 可以描述材料选择、事实与判断分离、引用检查及输出形式。Routine 则描述关注对象、频率、时间范围、停止条件与通知方式。两者还应能够独立停用：停止跟踪一个主题，不必删除相关研究方法；方法升级，也不该默认扩大订阅范围。

对后续智能体形态的影响，是用户可以从一个已验证任务逐步建立自动化。建设者需要保留每次运行的输入版本和方法版本，才能判断变化来自资料，还是来自流程修改。否则同一例行任务的结果变了，系统自己也无法解释原因。

14

多 Bot 协作：谁交接，谁承担完成责任

多角色协作最吸引人的承诺，是用户只需说明目标，不必自己分解所有步骤。Grok Bot 的公开文档描述了角色间消息、群组协作和任务交接能力。[G1] 但从产品分析看，“能够互相发消息”只是协作的前提，不能单独证明协调成本下降。

一份有效交接至少应保留任务范围、已完成部分、证据位置、未解决问题和下一步责任。假如研究 Bot 只把一句结论交给写作 Bot，后者可能无法判断结论适用时间；假如写作 Bot 把生成的页面交给发布 Bot，却没有共享范围，后续行动就缺少重要约束。

需要特别防止责任在角色之间消失。每个 Bot 都可以说自己完成了分工，但整体结果可能仍然无人验收。相对稳妥的组织方式，是让一个任务拥有明确负责人，其他角色提供可检查的子结果。负责人可以轮换，任务的目标与验收记录应保持连续。

交接内容	用途	缺少时的常见后果
输入与版本	确认双方谈的是同一材料	新旧结论混用
完成标准	判断子任务是否足够	无止境补充分析
权限与禁止事项	约束后续行动	把处理权限误作发布权限
证据与失败记录	支持复核和恢复	重复失败、重复操作
归属与截止条件	确定谁继续	任务在角色间悬空

角色数量增加时，应观察失败路径，而非只看顺利演示。一个角色无法读取资料，其他角色会等待、绕过还是凭记忆补写？一个子结果遭到纠正，哪些后续任务应撤回？这些问题能检验协作是否具有共同任务状态。

AbleWork 可以先把少数高频职责做稳，例如整理材料与检查引用，再考虑更复杂的角色编排。只有当独立分工能形成清楚的中间成果，并减少人的搬运或等待时，新增角色才值得维护。把同一个模型拆成更多名字，通常不能代替这项验证。

15

Grok Bot 的目标与代价：可委托的重复工作

Grok Bot 形态的目标，是让用户把一类跨工具工作持续交给固定角色，并逐步复用成功方法。它适合重复发生、输入输出相对清楚的工作。任务越稳定，角色记忆和流程复用越容易转化为实际收益；任务越模糊，用户仍需承担更多范围裁定与结果验收。

官方审批说明描述了行动前检查，以及允许一次、保存规则或拒绝等控制。它区分用户自己的请求与来自网页、邮件等外部内容，并提示审批只控制尚未发生的行动。[G3] 这些机制的产品意义，在于让长期角色仍然受具体行动范围约束，而非获得一个模糊的永久委托。

作者分析：例行化会放大正确方法，也会放大未被发现的错误。 一次错误筛选可能只漏掉一条记录；被保存为例行流程以后，同类遗漏可能持续发生。因此自动化前应先观察几个具有差异的样本，尤其是空结果、来源变化和需要人工判断的情况。

长期角色也需要退出机制。一个业务职责结束后，相关例行任务是否停止，已保存的登录是否仍然有效，旧角色能否继续收到外部事件，都影响管理成本。用户看到的是一个名字，系统需要管理的是一组随时间变化的任务、记忆与访问关系。

对后续形态的判断是：角色将成为用户组织工作的一种便利方式，但底层仍需可检验的任务和权限对象。角色适合承载交流习惯，不能代替工作契约。可移植的流程知识也可能比某个固定角色更有长期价值，因为它可以被不同执行者使用和修订。

AbleWork 与这种路径可以互补：个人或团队 Bot 负责完成工作，平台负责保存可分享成果及其来源，AutoWiki 负责将适合积累的材料编译成知识。是否值得进入正式知识，应由材料价值、更新成本和使用需求决定；不能因为某个 Bot 自动生成了结果，就默认永久收录。

16

Claude Tag：工作已经发生在团队讨论里

Claude Tag 从团队协作出发。官方介绍描述了在 Slack 频道中委托 Claude、异步执行并在原线程返回结果的方式。团队可以共同看到工作，也可以继续接上同一个讨论。管理员配置可用工具与范围，主动跟进属于可启用的行为。 [C1]

这一出发点对应着常见的组织成本：问题、决定和背景已经存在于讨论里，使用 AI 时却要另开窗口，再把前因后果复述一遍。结果完成后，还要搬回团队。把 Agent 放在工作发生的位置，可以减少这两次搬运，也让其他参与者看到委托究竟是什么。

作者分析：频道形态改变的是上下文的共同持有方式。 私人助手的对话主要由一个人维护；共享线程则允许不同成员补充事实、修正目标和查看结果。这有助于交接，但也意味着系统必须理解“谁正在提出什么要求”，不能把所有参与者的话都视作同等授权。

原处交付还有一个容易低估的收益：结果与最初问题留在一起。后来加入讨论的人，可以追溯为什么要做这项工作，哪些条件已经改变。一个独立生成的文档即使写得很好，如果丢失这段上下文，也可能在传播过程中被误解为正式决定。

然而频道天然混合事实、猜测、玩笑和暂定意见。一个成员提出方案，不代表团队已经通过；消息获得几个表情，也不一定等于批准行动。产品如果希望主动跟进，就需要识别未决问题与正式承诺，并给用户一个容易纠正的表达方式。

AbleWork 与此最直接的对应是“在对象旁工作”。研究员阅读报告、查看主题或指标时发起委托，结果回到相关对象，其他有权限的人能够继续使用。它可以借鉴线程协作，但研究对象往往比一段讨论存在更久，因此需要额外保留对象版本和知识更新关系。

17

原线程交付，让异步工作仍然可理解

Claude Tag 使用文档说明，频道任务在临时沙箱中运行，结果回到线程；执行环境的生命周期与用户电脑不同。文档也描述了任务进展的可见表达。这些机制用于支持团队异步委托，而不要求发起人一直守着执行过程。 [C2]

FIG. 04 / 机制示意

在原讨论中委托与交付



机制示意 · 箭头表示工作关系，不表示耗时或性能

团队在共同可见的位置接续工作；执行范围由配置与授权约束。依据 C1—C3 简化。

异步工作带来的首要问题是预期。用户离开之前，需要知道系统已经接受什么；回来之后，需要知道已经完成什么。一个模糊的“处理中”不足以支撑长任务，因为它无法区分正常等待、输入不足和真正失败。对用户有用的状态，应说明下一步依赖谁或什么事件。

设想团队在频道里发现一次指标异常，请 Agent 调查。调查可能发现数据采集延迟，也可能发现代码变化。结果应该分别列出观察、解释、证据和建议；如果需要修改生产系统，应把这项行动单独呈现。把分析写进线程，不等于已经获得修改所有相关系统的权限。此处是通用情境推演。

原线程也不必承担所有详细内容。长报告、代码变更和数据表适合放在独立成果中，线程保留摘要、入口和待决事项。这样既能保持上下文连续，又不让整个频道被大段输出淹没。不同载体各自承担适合的阅读任务。

这给 AbleWork 的启发是，任务状态、成果和讨论可以相互链接，但不必合成一个无限延长的聊天记录。报告承载完整论证，主题承载长期问题，任务承载执行进度，讨论承载协商。它们之间的关联应清楚，才能让用户既看见进度，也找到可长期使用的结果。

18

团队记忆：共享上下文仍需要范围

Claude Tag 公开材料把工具、记忆与管理员定义的范围联系起来，并提供专门的记忆说明。共享频道上下文与私人对话不是同一类访问关系。[C1] [C4] 本文据此讨论团队记忆的设计问题，不假定任何看到频道的用户都能读取组织全部资料。

团队记忆最有价值的部分，常常是那些没有写进正式文档的工作背景：某个项目为什么延后，某个数据字段为何暂时不用，一项决定在什么条件下成立。保存这些背景能减少重复解释，但也容易把临时意见长期固化。

作者分析：团队记忆需要区别“有人说过”与“团队采用”。 前者是讨论事实，后者是组织状态。系统应尽量保留判断的提出者、适用条件和最后确认时间。否则一条旧消息经过摘要，再经过另一次回答，会逐渐失去原本的不确定语气。

跨频道复用还涉及目的变化。销售讨论中的某个客户细节，未必适合进入工程频道的概括；即使两个频道都能读取同一文档，也不代表所有推导结果都适合公开。权限检查需要发生在读取和输出的位置，不能只在最初接入时检查一次。

从知识管理角度看，频道记忆适合帮助团队接续工作，却未必等于正式知识库。正式知识需要解决冲突、旧版本、引用和失效问题。团队可以把成熟的讨论转成文档或知识条目，但这一过程应有明确动作，而非让模型默默把一切讨论变成默认共识。

AbleWork 的知识编译可在这里承担互补角色：把适合积累的讨论成果与源材料比较，形成有出处的知识；把仍有争议的观点保留为候选或带署名判断。学习共享记忆的便利时，也应保留知识获得正式地位所需的检查。

19

Agent 身份：谁的工具，谁的行动

Claude Tag 的身份文档区分频道内的服务账号身份与个人账户路径。频道工作主要使用管理员配置的访问；个人连接器可在用户允许后用于其自己的任务，其他成员的请求不会因此继承个人连接器权限。凭据由相应代理机制处理，不能简单理解为发起者把全部账户交给频道。[C3]

这个区分使组织能够解释行动归属。谁提出了任务、哪个 Agent 执行、使用哪种账号、最终写入哪个系统，是不同字段。若只保留“Claude 做的”，事后很难判断是需求错误、配置错误还是执行错误，也难以定位谁有权纠正。

作者分析：共享 Agent 的身份设计，应先于大范围工具接入。 一个组织可能希望所有频道成员都能让 Agent 查询某个公共数据源，但只有少数人可以提出变更。读权、发起权、执行身份和发布权可以不同。把这些全部绑定到一个万能服务账号，虽然接入方便，却增加解释与治理成本。

还需要考虑离职、换组和项目结束。某个长期任务原来由有权成员发起，后来成员关系改变，任务是否继续？某份结果原来可见，后来来源撤回，已经缓存的摘要怎么办？这些问题超出了聊天界面，却是团队持续 Agent 的日常问题。

在研究平台中，对应的身份链可以是作者、任务发起者、执行者、发布者和读者。作者提供材料，不一定批准所有衍生用途；读者可以提出问题，不一定能够修改源报告；执行者可以生成候选，不一定能够写入公共知识。把这些角色写清楚，能减少后续规则的临时补丁。

身份设计的最终目标是让用户理解发生了什么，而不是展示更多技术名词。一次结果交付可以简明说明“基于哪些有权材料、完成了什么、由谁确认发布”。更细的审计记录留给需要追查的人，日常体验保持清楚即可。

20

Claude Tag 的目标与代价：团队少搬运一次上下文

Claude Tag 形态的目标，是让团队在原本协作的位置使用 Agent，并共享任务过程。它最直接的优势可能不是让单个回答更聪明，而是让调用、纠正和交付进入同一段工作关系。这对多人接力、跨职能调查和需要共同确认的任务尤其有意义。

代价则来自共享空间。一个私人助手多说几句，主要影响一个人；频道 Agent 频繁发言，可能影响整个团队。主动跟进应以尚未解决的问题、明确责任和有用变化为依据，而非看到新消息就参与。用户需要能够调整它何时回应，以及哪些情况适合静默准备。

第二项代价是讨论中的歧义。多人可能提出冲突要求，任务负责人也可能中途变化。系统应暴露冲突，并保留当前采用的目标。不能用最后一条消息机械覆盖整个工作，也不能永远固守最初指令，让后续合理修正无法生效。

第三项代价是组织配置。频道范围、连接器、服务账号和预算需要维护。如果一项简单任务需要频繁找管理员调整配置，便利性就会受损。因此应先选择工具需求稳定、权限范围清楚的频道，形成可靠实践，再逐步扩展。

作者判断：嵌入协作空间会是一种长期存在的形态。 它利用已有交流习惯，但不自动解决正式成果管理。讨论结束以后，团队仍需要知道哪个版本可用、哪些决定已通过、哪些结论已经过期。线程与文档、任务与知识之间的连接，将决定这种形态能承担多长的工作周期。

AbleWork 可以吸收原处协作和共同可见的优点，同时围绕报告、主题和指标保留长期对象。它不必变成一个全天候在社区发言的虚拟同事。对研究工作而言，适时出现的结果、明确的来源和可继续追问的上下文，可能比更强的社交存在感更有价值。

21

Super-agent：从事件不断到来的工程工作出发

DeepSeek 公开的 dsh-superagent-prototype 面向一个 GitHub 仓库的编程协作，可接收 GitHub 及可选飞书事件。项目明确声明它是研究原型，不是官方生产产品；公开代码与内部版本、设计文章不完全相同，部分设计仍属未来方向。[D1]

它要解决的问题与普通问答不同。工程工作可能由新问题、审查意见、CI 结果和人的追问不断推动。一次模型调用结束，并不意味着任务结束；等待外部事件也不意味着系统失败。持续任务系统需要把这些情况区分开，才能减少人反复唤醒、解释和追踪的负担。

作者分析：这种形态把“工作状态”提升为独立对象。 执行会话只是处理工作的一个载体。会话可能重启、换执行者或等待很久，任务仍应保有目标、授权、结果和后续输入。若状态只存在于最后一段对话里，任何中断都可能迫使系统重新猜测。

工程任务适合暴露这类问题，因为它有较明确的外部对象：Issue、分支、PR、审查、检查结果。用户可以看到代码是否提交，检查是否通过，变更是否合并。这些对象为持续工作提供了落点，也让失败更容易被定位。

研究任务的外部对象通常没那么整齐。新报告、新数据和观点变化可能来自不同来源，结束条件也更开放。因此 AbleWork 若借鉴这种任务组织，不能只复制工单字段；需要明确研究对象、材料版本、待验证问题和后续触发条件。

本报告关注其可复用的组织思想：事件如何成为任务、授权如何随任务保留、等待如何表达、恢复如何避免重复行动。不会把“Super-agent”这个名称当作已经实现全自主工作的证据，也不会用公开研究代码替代生产成熟度评价。

22

从事件到任务：看见变化，不等于可以行动

原型的领域文档区分 Source、Trigger、Thread、Eval、Mandate、Ticket 与 Assignment。事件进入后可能成为已有任务的上下文，也可能经过评估成为新任务；接受任务时记录授权依据与范围。执行者通过派工关系承接工作，结果回到相应渠道。 [D2]

FIG. 05 / 机制示意

事件进入持续任务



机制示意 · 箭头表示工作关系，不表示耗时或性能

已有任务可接收相关上下文；新工作需要授权判断。依据 D2、D3 简化，不展示全部分支。

这套区分的意义，在于减少两种混淆。第一种是把任何新消息都当成新工作，产生重复任务；第二种是把上下文材料当成新的授权，使一个本来有限的任务不断扩大。事件可以提供信息，而不改变工作范围。

设想代码任务正在等待 CI。检查结果到来，应继续已有任务；另一位成员在相邻讨论中提出新功能，未必属于当前授权。如果系统只根据主题相似度追加工作，任务很容易无边界增长。更合理的做法，是把相关性与工作归属分开判断。

任务授权还应能够解释。系统不只记录“被允许”，还要知道是谁、依据哪条规则、允许到哪里。这样当政策变化或任务恢复时，才有依据判断是否继续。授权是一项工作事实，不应完全依赖执行者每次阅读聊天历史后的印象。

对 AbleWork 的对应，是新材料可以触发分类或候选整理，但不能因为它出现了，就自动修改作者观点或扩大共享范围。已有研究问题可以吸收相关材料；新的研究目标则可能需要新的委托。事件丰富，不代表行动权限也随之增长。

23

等待、失败与恢复：让人知道为什么停在这里

原型执行文档明确区分工作中、等待触发、等待人工回答、等待审查、受阻和失败等状态，并描述了输入持久化、重试和重启后的恢复。恢复时需要检查已有目录、版本历史及外部结果，以避免重复外部行动。[D3]

作者分析：持续任务系统最重要的状态信息，是下一步依赖。“没有输出”可能表示正常等待审查，也可能表示模型服务故障，两者需要不同处理。把它们统称为忙碌或失败，会导致用户反复催问，或者误以为任务仍在推进。

重试也不能只等于再次发送同一提示。若前一次已经创建 PR，但回执丢失，直接重跑可能创建第二份；若已经写入报告，只是在通知时失败，应补通知而非重新生成。恢复必须先确认哪些效果已经发生，再选择剩余动作。

持续任务通常有三个不同的记录：打算做什么、系统实际尝试了什么、外部世界最终接受了什么。它们不一定同时成功。任务记录应能保留差异，例如“文件已保存、发布未完成”“修改已提交、审查待定”。这种表达比一个总成功标记更有诊断价值。

在研究编译中，候选生成可能消耗较大成本。如果合并时遇到版本变化，保留候选并重新比较，通常比从头生成更合理。但旧候选不能直接覆盖新基线，仍需要确认它针对的材料与现有知识是否一致。恢复是在保留成果的同时重新检查条件。

对未来 Agent 的影响，是“会继续”将逐渐包含更多工程含义：保存状态、识别外部效果、接受新输入、正确等待以及有限重试。模型能长时间推理只是其中一部分。用户真正需要的是工作不中断，且中断后能够解释和恢复。

24

低打扰与执行边界：原型的启发也有适用范围

Super-agent 的行为原则强调尽量少占用人的注意力，只在明确请求或团队规则允许时行动，并在有可用结果或需要决定时发言。[D4] 因此不能把“低打扰、有授权的主动性”写成 AbleWork 独有的发明；它是不同持续工作系统共同面对的问题。

原型也明确说明，任务目录与命令包装建立的是默认行为，不构成安全边界；执行器以操作者的操作系统用户运行。[D3] [D5] 独立目录可以减少文件混淆，独立会话可以减少上下文混淆，但无法据此推导对宿主资源的强隔离。

作者分析：可靠比较必须把隔离拆成具体对象。 是对话上下文分开，文件目录分开，进程权限分开，凭据分开，还是网络出口受限？每一种保证都有不同作用。把它们统称为“沙箱”，容易使读者高估部署后的安全属性。

研究原型的价值，在于公开了可讨论的概念和取舍。它允许建设者看到为什么任务与会话分开、为什么上下文不应扩大授权、为什么失败需要持久化。生产化还要结合部署环境补充身份管理、资源限制、凭据控制和维护流程，不能省略这一步。

对 AbleWork 来说，可以借鉴低注意力成本的工作原则和任务组织，而保留自身的多用户材料边界。研究平台存在跨作者成果和共享读面，不能把单仓库工程助手的信任假设原样扩大到整个平台。

判断一种架构是否适用，应该先写出它假定哪些人可信、能访问哪些资源、错误可能影响多大范围。假设相近时，机制容易复用；假设不同，就需要调整，而不是争论某种形态抽象地更先进。

25

Super-agent 的目标与代价：减少人维持任务的工作

Super-agent 形态的目标，是让连续事件推动工作，人主要处理需要判断和验收的部分。它试图减少的负担包括派工、补上下文、催进度、处理中断以及把结果搬回原讨论。这些工作在复杂任务中经常比首次提问耗费更多注意力。

其代价是状态管理与运营复杂度。系统需要知道哪些输入已经处理，哪些任务等待外部事件，哪些失败可重试，哪些必须由人解决。只有一个简单任务时，这套结构可能显得过重；多个任务持续交错时，它才更容易体现价值。

作者判断：任务运行系统可能成为多种 Agent 产品的共同底座，但不必成为用户直接面对的界面。 用户仍可以在聊天、频道或研究对象旁委托。底层把授权、状态和回执保存下来，前台只呈现与用户有关的进度。工单概念可以存在于实现中，不必要求每个研究员先学会管理工单。

这种形态也不会自动保证结果质量。CI 通过能证明某些检查满足要求，不等于变更符合全部业务目标；任务状态为完成，不等于用户已经采用结果。研究场景更需要区分工程完成与论证质量，否则系统会把“成功生成”误认为“结论可靠”。

对 AbleWork 的合理结合，是在少数真正需要跨天等待的研究任务上引入明确状态：等待新材料、等待作者确认、等待来源可读、等待审查。短问题仍可快速回答，避免把所有交流都变成长任务。

由此得到一条建设顺序：先发现真实中断，保存必要状态，再引入恢复与事件推进。不要为了拥有“持续 Agent”而先制造复杂工单流程。系统应该承担原本由人维持的连续性，而不是新增一套必须由人持续维护的管理界面。

26

横向比较一：谁是工作的中心

五个案例的差异首先来自工作中心，而非界面大小或模型名称。中心对象决定上下文围绕什么累积，任务如何归属，完成结果应该放在哪里。下面是基于前文事实所作的作者归纳，并非产品排他定义。

形态	主要中心	用户首先希望减少什么	结果通常回到哪里
Muse	个人与目标	零散事务的协调	个人任务、计划与成果
Grok Bot	角色与流程	反复配置和跨工具操作	角色对话与实际工具
Claude Tag	团队频道与线程	上下文搬运和接力解释	原讨论与链接成果
Super-agent	获授权的持续任务	派工、等待与恢复	任务渠道及工程对象
AbleWork	研究对象与知识	找材料、核版本和重复研究	报告、主题及知识读面

中心对象不同，默认优先级也不同。个人代理更需要理解偏好；团队 Agent 更需要理解共同可见的范围；任务系统更需要保持状态准确；知识平台更需要解释结论来源和更新时间。这些能力可以组合，但不能靠增加一个入口就自动获得。

一个实用选择方法，是观察用户最常说的失败抱怨。“每次都得重新解释我是谁”偏向个人关系；“事情在几个软件间搬来搬去”偏向流程；“同事不知道已经做了什么”偏向共享协作；“一等外部结果就断了”偏向任务接续；“别人做过的研究找不到或不敢用”偏向知识管理。

产品应首先对准其中一个高频负担。否则所有能力都做一点，用户仍不知道什么时候应该使用它。AbleWork 的主要承诺应围绕研究成果进入后续工作展开，再解释就地委托和持续关注怎样服务于这个承诺。

这种比较也避免了不必要的替代叙事。一个研究员可以使用个人 Agent 安排行程，用角色 Bot 整理材料，在团队频道讨论，再把正式成果放进研究平台。多个形态能长期并存，关键在于交接时是否保留身份、权限和来源。

27

横向比较二：触发、准备、通知、行动

“主动性”不适合只用开关或高低分来比较。一次持续工作至少包含发现触发、判断关联、准备结果、选择呈现时机和执行外部行动。某系统可以积极准备但很少通知；另一个系统可能频繁回复，却没有持续执行能力。两者都可能被营销为主动 Agent。

FIG. 06 / 机制示意

把主动性拆成具体动作



机制示意 · 箭头表示工作关系，不表示耗时或性能

每一步都有独立条件；准备可以积极，通知可以克制，外部行动仍需明确授权。作者分析。

作者分析：应把主动程度与后果分开。 读取已授权资料、整理内部草稿、提醒截止时间和对外发出承诺，涉及不同风险。用户允许系统每小时检查变化，不等于允许它每小时在公共频道发言。用户允许准备研究建议，也不等于允许修改正式观点。

触发本身也需要语义。定时器到点，表示“可以检查”；新文件出现，表示“有新输入”；某指标越过阈值，表示“可能值得关注”。它们都不直接证明用户必须采取行动。系统应该把触发解释为候选工作，再依据既定目标与当前事实决定如何处理。

低打扰设计可以有多种实现：重要变化即时通知，一般变化汇总呈现，重复变化不重写，尚无足够证据的变化留作候选。这里提出的是通用设计建议，不表示五个产品都采用同样规则。规则应让用户能够理解和修改。

AbleWork 当前的订阅检查与晨报准备体现了其中一条路径：在明确关注范围内检查事实，有需要时组织结果，用户回来可读。它与开放目标的自主发现不同。把这两者分开，既承认当前主动工作的价值，也保留未来能力需要验证的空间。

28

横向比较三：持续的是任务，还是机器

持久云端电脑、临时沙箱、任务会话和后台定时进程，都是可能的运行形式。它们不能直接排列成先进程度。持久环境便于保存工作文件和登录状态；临时环境便于限定单次任务的生命周期；明确的任务记录则可以跨不同环境保持工作连续。

运行选择	可能收益	需要额外解决的问题
持久环境	接续方便、少重复配置	状态污染、资源竞争、过期登录
临时环境	清理清楚、任务范围易表达	成果保存、环境重建、状态恢复
常驻会话	上下文延续直接	历史膨胀、错误累积、成本
事件唤醒	空闲期少占资源	事件去重、恢复上下文、漏触发

作者分析：真正需要持久的是有业务含义的状态。 例如目标、授权、输入版本、已经发生的外部效果和待决事项。机器可以更换，会话可以压缩，只要这些状态能够正确恢复，用户仍会认为工作在持续。

反过来，一台一直运行的电脑也可能没有持续工作能力。若任务结束条件不明确，文件混在一起，外部操作没有回执，进程常驻只会让问题积累更久。判断“持续 Agent”时，应检查中断后行为，而不是只看后台是否有进程。

研究平台的运行安排更应遵循工作性质。短摘要可以是独立任务，长报告需要可恢复产物，主题跟踪需要事件与时间条件，正式知识合并需要短而明确的写入阶段。不同阶段使用不同运行方式，通常比要求一个永不结束的会话承担全部工作更容易管理。

因此 AbleWork 与其他形态结合时，应优先约定任务与成果接口，再决定执行器部署方式。可替换的执行者需要清晰的输入和回执；只共享一个工作目录，往往不足以形成可靠的跨系统协作。

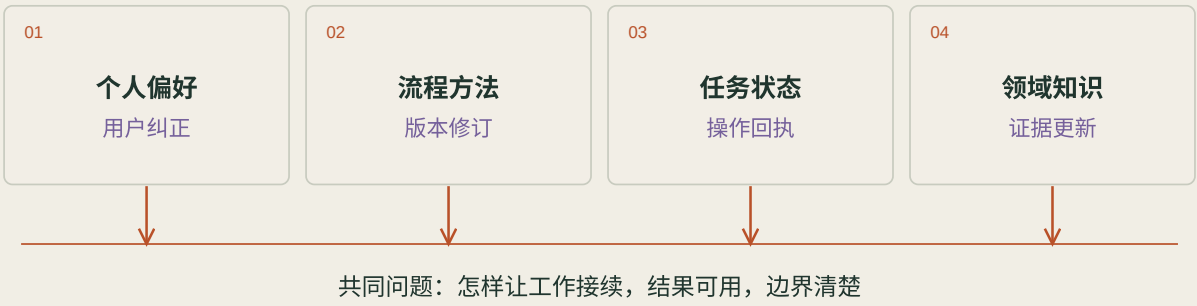
29

横向比较四：记忆、流程与知识分别更新

同样叫“记住”，产品可能保存完全不同的东西。个人偏好服务于交流，角色经验服务于重复工作，线程记忆服务于团队接续，任务记录服务于恢复，领域知识服务于后续判断。它们有不同的正确性标准和失效方式。

FIG. 07 / 机制示意

不同记忆需要不同更新方法



对象不同，正确性与失效方式不同；召回只是其中一步。作者分析。

偏好改变时，通常由用户直接纠正；流程改变时，需要检查新旧方法与适用条件；事实改变时，需要来源和时间；任务状态改变时，需要执行回执。若这些内容都由模型自行重写成一段综合摘要，就很难回答某项变化为什么发生。

作者分析：记忆系统的成熟度应看更新与撤回，而不只看召回。 找到旧信息只是第一步。系统还需要知道旧信息是否仍然有效、与新证据是否冲突，以及谁有权修改。越接近正式组织知识，对这些要求就越高。

在研究场景中，“某分析师当时认为需求会改善”与“需求已经改善”尤其需要分开。前者可以长期保存为带署名的历史判断，后者应由对应期间的数据支持。摘要如果删掉时间和条件，就可能把观点变成事实。知识编译因此必须保留来源语义，而非仅追求压缩率。

AbleWork 可以让几类存储共存：个人方法帮助工作，任务状态帮助恢复，报告保留完整论证，知识页组织可更新的结论。它们之间通过来源与版本关联，但不应彼此替代。最终读者看到的，应是当前问题需要的内容，以及足够判断其可靠性的依据。

30

横向比较五：目录、工单与向量检索可以共存

“为什么用目录树而不用工单机或向量记忆”这个问题需要改写。目录、工单和向量检索解决不同问题。目录帮助人和 Agent 浏览、编辑与引用稳定对象；工单描述工作状态与责任；向量检索帮助发现语义相关内容。三者可以共同存在于一个系统中。

组件	主要回答的问题	单独使用的不足
目录与文档	内容在哪里，如何组织	不自动管理并发与权限
任务系统	谁在做，等待什么	不自动形成可靠领域知识
向量检索	什么内容可能相关	相似不等于有效或有权使用
结构化数据库	对象、关系和状态是什么	需要明确建模与维护

目录的优点是直观、可携带、易于人工审阅。它适合让知识对象具有稳定位置，也方便一般工具读取。但文件存在不等于内容正式，文件名也不能替代身份授权。多人写入时，还需要版本控制、冲突处理和发布流程。

工单适合持续任务，却不一定适合成为研究员的主界面。用户希望围绕主题工作，系统可以在背后建立任务。向量检索可以增强发现，但应在授权范围内运行，并在最终引用时核对来源、版本和可见性。检索得分不能成为真伪判断。

作者分析：AbleWork 的选择应描述为面向不同对象组合这些机制。 它既有文件形态的知识与个人工作区，也有结构化社区、任务和知识数据。把项目写成“目录战胜数据库”或“知识库战胜工单”，会误导对真实工程的理解。

选型时可从一次失败反推。如果用户找不到内容，先改善索引与检索；如果找到的是旧结论，改善版本与失效机制；如果工作没人接，改善任务归属；如果结果越权，改善授权与发布。换一种存储形态，通常不能一次解决所有问题。

31

横向比较六：权限沿着什么传递

一个 Agent 任务通常经过委托、取材、执行、交付和后续复用。每一步都可能改变参与者或可见范围。用户有权读取某份资料，不表示整个团队有权看到由它生成的摘要；Agent 有权创建草稿，也不表示它有权正式发布。权限需要沿着工作路径检查。

FIG. 08 / 机制示意

权限随工作路径反复检查



机制示意 · 箭头表示工作关系，不表示耗时或性能

相关性不授予权限；读取、执行、发布与后续消费分别检查。作者分析。

前文案例给出了不同实现：个人代理围绕个人连接与行动审批，角色产品围绕账户环境与行动控制，频道 Agent 围绕管理员配置及个人连接器例外，任务原型围绕工作授权，AbleWork 围绕材料与结果的身份、版本和读权。这里比较的是主要约束对象，不能据此宣布谁更安全。 [M3] [G3] [C3] [D2] [A1]

作者分析：权限不能从“相关”自动推导。 一份材料与任务相关，只说明可能有帮助；一个人参与讨论，只说明他在提供上下文。系统还需分别确认可读取、可使用、可修改和可分享。相似度、成员关系和任务归属都不能单独替代授权。

共享结果尤其需要注意信息变换。去掉姓名的摘要仍可能包含足以识别对象的细节；多个低敏感材料组合后也可能产生更敏感的推断。正式发布前应关注结果实际包含什么，而不是只看每个输入的标签。

对于 AbleWork，最重要的产品承诺之一应是：别人能够复用被允许共享的成果，而原作者不必因此交出整个私人工作区。公共可读内容、个人研究过程和候选提交可以分开。这样分享的是具体成果，不是默认开放作者全部记忆。

32

横向比较七：结果究竟留在哪里

Agent 完成工作以后，结果可能是一条回答、一个文件、一项业务系统更新、一份 PR，或者一个知识版本。不同结果有不同的复用方式。把它们统称为“产出”，会掩盖用户下一步还需要做多少工作。

结果形式	适合用途	需要补充的长期信息
对话回答	快速理解、临时判断	来源、时间、后续纠正
文件或页面	完整阅读与分享	作者、版本、稳定入口
系统写入	直接改变业务状态	操作回执、恢复办法
PR 或任务结果	受审查的变更	检查、批准与最终状态
知识条目	跨任务持续复用	来源、适用条件、失效关系

作者分析：交付不是最后一句“完成了”，而是结果进入了可使用的位置。 用户如果还需要追问文件在哪里、哪一版是最新、能不能分享，系统的交付就没有完成。一个可见链接也不足以说明全部完成：链接可能指向临时文件，或对目标读者不可读。

后续复用的成本与结果结构有关。只有一大段文本，读者需要重新判断哪些是事实、哪些是假设；有明确来源和时间的报告，更容易核对；有更新关系的知识条目，更容易在新材料出现时修订。结构带来维护成本，但也能减少后来者重复推导。

AbleWork 的定位在这里比较清楚：让成果拥有可见、可追溯、受权限约束的位置，再进入后续阅读和问答。它不能替代所有个人 Agent 的执行能力，但可以承接不同执行者产生的研究成果。是否值得编译为知识，应由后续使用需求决定。

因此系统组合的接口应围绕成果，而不只是围绕消息。至少应能传递成果标识、版本、作者、来源、可见范围和状态。这样一个 Agent 离开后，其他人和其他 Agent 才能继续工作，而不会只剩一段无法验证的“已完成”描述。

33

横向比较八：成本与注意力要一起计算

持续 Agent 的成本不只有模型费用。还包括执行环境、存储、连接维护、失败恢复、人工检查和用户注意力。一个系统调用次数很少，却需要人反复修正和搬运，未必便宜；另一个系统调用更多，但稳定替代了一段耗时工作，也可能更有价值。

本报告建议使用一项解释性账本：**总工作成本 = 系统资源 + 人工准备 + 人工审核 + 失败恢复 + 被打断的注意力。** 这不是已经测量的公式，也不把不同单位直接相加；它提醒建设者至少记录这些成本项，再按实际业务方法估计。

主动检查需要尤其谨慎。高频唤醒不一定需要每次调用模型，来源没有变化时可以保留检查记录。发现变化以后，也不一定需要完整重编译。阅读导航可能只需摘要；重要结论变化可能需要精读、比较和人工判断。处理深度应与预期用途相称。

角色协作也有额外成本。每一次交接都可能重复上下文和验证，如果任务本身很短，分工反而增加开销。适合并行的部分应能独立验收，并且其等待时间或专业差异足以抵消协调成本。不能以角色数量作为先进程度的指标。

对 AbleWork 而言，日常梗概与完整编译分路，正是需要结合用途理解的选择。它不能只用“更便宜”解释，因为便宜可能牺牲细节；也不能只用“更完整”解释全量编译，因为许多材料并不值得长期维护。应观察后续问题是否真的需要那些细节。[A1]

评估时可先采集少量真实任务的全程记录：用户开始准备到最终采用之间经过了什么，而不只记录模型运行时间。若没有原流程基线，就报告观察到的行为变化，暂不宣称效率提高了某个百分比。可解释的有限证据，比漂亮但无依据的提升数字更有用。

34

横向比较九：质量应该在哪些地方被检查

Agent 系统可以在输入、候选、正式写入、发布和后续消费几个位置检查质量。不同检查回答不同问题。格式合法，不能证明事实正确；事实有来源，不能证明来源足以支持结论；执行成功，也不能证明用户接受结果。

FIG. 09 / 机制示意

检查靠近它能控制的后果



机制示意 · 箭头表示工作关系，不表示耗时或性能

格式、来源、语义、写入与可见性是不同检查，不由一个成功标记替代。作者建议。

输入检查关注材料是否完整、身份与版本是否明确。候选检查关注引用、结构、冲突和适用条件。正式写入检查关注是否基于当前状态、是否发生重复或覆盖。发布检查关注目标读者和可见范围。后续消费则要重新考虑来源是否仍有效。

作者分析：检查应靠近它真正能够控制的后果。 生成前可以防止读入不该使用的材料，但无法预先知道最终文本是否歪曲条件；发布前可以检查结果可见范围，却不能补回已经丢失的原始依据。只在最后加一个“审核 Agent”，往往不足以代替前面各阶段的约束。

独立复核也需要独立证据。如果两个模型只看同一段错误摘要，它们意见一致并不意味着可靠。让复核者能够看到原材料、具体引用和需要判断的争议，比让它简单评价“是否合理”更有价值。复核任务本身也应该有明确标准。

AbleWork 历史编译的回执可以证明比较、校验、提交和发布之间存在实际连接；它不能证明所有知识已经逐句由专家确认。因此报告把结构可追溯与语义质量分别说明。后续质量建设应从高影响、常复用的内容抽检开始，逐步形成纠错记录。[A1]

35

五种形态怎样组合，而不让责任丢失

一种可能的工作组合是：个人代理帮助用户明确长期关注，角色 Bot 执行重复整理，频道 Agent 支持团队讨论，持续任务系统处理等待与恢复，知识平台保留正式成果。这个组合是作者提出的设计模型，并非五个产品已经建立互联。

FIG. 10 / 机制示意

一种可能的组合，而非已建成的互联



跨形态交接携带成果、版本、来源、权限和责任。此图为作者提出的组合模型。

组合的难点在边界。个人偏好不应自动成为团队规则；团队讨论不应自动授权外部发布；一个执行任务成功，也不意味着知识条目已经可供所有人使用。每次跨系统交接，都需要说明结果的性质与权限，而不能只传递最终文本。

最小可行交接可以很朴素：一份成果、一个稳定版本、来源清单、当前状态、允许的用途和负责确认的人。只有当交接频率与复杂度增加时，才值得建设更完整协议。先有真实接续，再做通用抽象，通常更容易避免无人使用的接口。

研究平台应保有对正式知识的最终控制。外部 Agent 可以生成材料和候选，平台依据当前规则决定如何纳入。这样执行者可以替换，已有知识不会跟着某个对话或某家工具消失。反过来，平台也应提供清楚的读接口，让不同 Agent 使用被允许的知识。

对 AbleWork 的近期方向而言，这比同时复刻四种产品更具体：把成果接入、版本核对和后续消费做好，再选择一个高频职责引入流程复用。每新增一种外部执行形态，都应能够证明交付更省心，且没有使作者与读者的边界变得模糊。

36

AbleWork：研究工作的知识托管与边界治理

AbleWork 的一句话定位可以写成：**它是研究协作场景中的知识托管与边界治理平台，让人和 Agent 的成果可以积累、共享、更新与追溯。** 平台中的 Able 承接报告、主题和指标旁的委托，并在已建立的关注范围内持续准备结果。知识能力与工作组织共同服务于研究流程。 [A0] [A1]

这个定位比“比个人 Agent 多一层”更清楚。研究员的问题不是缺少另一个会说话的工具，而是材料、判断和成果不断增加，下一次工作仍可能重新开始。有人已经研究过，其他人却不知道；找到了报告，又不确定是否为最新版本；愿意分享结论，却不希望开放私人工作过程。

AbleWork 因此围绕几个稳定对象组织工作：来源材料、研究成果、主题、任务、知识和参与者。对象之间保存关系，用户可以在工作发生的位置委托，而不是每次从空白聊天开始。报告保持作者归属，衍生的梗概与知识保持来源联系，读取时继续考虑权限。

作者分析：平台的价值来自成果的第二次使用。 第一次产出可以由很多工具完成；如果成果能够被另一个人用于后续问题，且保留版本和来源，平台才开始承担独特责任。这里的“第二次”是产品判断，不是限定必须恰好两次访问。

这也决定了它不需要以社交存在感为中心。Able 可以主动准备材料，也可以在需要决定时提醒，但不必为了像同事而持续发言。研究员应更容易找到有用结果，而不是多管理一个需要回应的角色。

当前完整度应据实描述：社区发布、梗概调用、历史完整编译以及有边界的主动准备已有运行依据；个人候选进入公共层的正式采纳仍缺完整实例证据。项目已经在使用，不等于每条设计路径都已完成。 [A1] [A2] [A3]

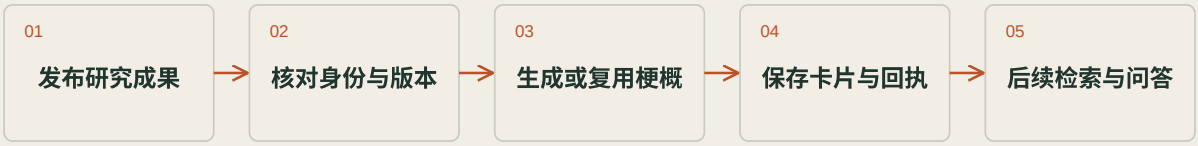
37

主路径：从研究成果，到后续问题的输入

AbleWork 当前社区增量采用分类、梗概与主题导航。用户或其 Agent 保存、发布成果后，宿主核对来源身份、版本与读权，后台生成或复用梗概，保存报告卡及回执；后续检索与问答读取时再次检查来源条件。这是当前日常路径。 [A1]

FIG. 11 / 机制示意

AbleWork 当前日常路径



机制示意 · 箭头表示工作关系，不表示耗时或性能

消费前继续核对来源版本和读权；当前默认是梗概导航，非全量完整编译。依据 A1。

这条路径的重点是保持引用关系。摘要不应脱离源报告成为无主文本。源报告更新后，旧摘要不能继续冒充当前版本；来源不再可见时，缓存不能绕过新的权限状态。报告卡、来源版本与读取条件之间需要可检查的一致性。

作者分析：梗概的任务是帮助发现和理解材料，而不是替作者完成全部论证。 它可以告诉读者材料讨论什么、哪些条件值得关注，并帮助问答定位可用内容。需要细节或重要判断时，应回到原文。把梗概当成完整知识，会放大压缩时丢失的信息。

这条主路径已经足以构成有价值的应用循环：成果发布以后，其他任务能够实际读入。它并不要求每篇材料先变成大量概念页，也不要求所有用户先参与复杂知识维护。较轻的入口有助于让共享成为日常行为。

但这一价值仍需继续验证深度。来源被调用，说明资料进入了任务；回答得到交付，说明系统完成了输出；读者是否认可、是否据此采取行动，是另外的结果。AbleWork 当前材料可以证明前两者的具体样本，尚不能把它们全部写成最终研究采用。

38

运行规模：说明有人使用，也保留统计边界

作者运行核查覆盖的近期窗口为北京时间 2026 年 10 月 1 日 23:37:37 至 10 月 31 日 23:37:37，左闭右开。按可识别个人主体去重，并排除测试、后台同步及归属不明活动。这是一项保守使用汇总，不是全平台月活普查。 [A1]

观察项	汇总结果	可以说明什么
真实业务使用者	19 人，其中 18 人非作者本人	已有内部自然使用
非本人作者	13 人	成果来自多位使用者
非本人发布成果	50 个不同报告或站点	已发生持续内容交付
调用社区梗概的问答	11 单、7 人	知识输入进入后续任务
当前版本梗概	301 / 318 个在架来源	日常导航已有实际覆盖

50 个成果按作者与成果标识去重，多版本不重复计数。11 单问答表示来源记录中调用过社区梗概，不等于 11 个正确答案，也不代表每份报告都被使用。301 / 318 对应核查时点的在架来源，剩余 17 个被明确延后，不应写成隐含成功。 [A1]

这些数字支持“项目已投入使用”的表述，也说明平台不只是作者个人演示。但它们不足以支持留存率、研究效率提升或决策质量改善。相关指标需要更完整的分母、基线和用户采用记录，不能从发布数量推算。

作者分析：当前最值得追踪的是从发布到有意义复用的转化。 哪些成果被跨人调用，哪些问题因此少做了重复查找，哪些摘要需要回到原文纠正。这个问题比单纯增加报告数量更接近平台定位。

后续统计应继续把人、Agent 和后台任务分开。代人执行的 Agent 归到相应主体，不能另算新增用户；重试不算新成果；知识页不算输入报告。清楚的计数规则有助于避免系统越自动化，指标看起来越繁荣，却无法对应真实使用。

39

脱敏案例：一人的新版研究进入另一人的问题

运行核查记录了一条跨作者路径：一位研究者发布并更新报告至 v2，平台随后生成对应版本的梗概；两天后，另一位使用者提出相关后续问题，问答来源记录包含这张 v2 卡，版本与内容指纹能够对应。这里保留流程，省略业务对象和用户身份。 [A1]

FIG. 12 / 机制示意

跨作者复用的已观察路径



机制示意 · 箭头表示工作关系，不表示耗时或性能

脱敏运行样本：证明新版成果进入另一人的任务输入，不证明答案已获用户接受。依据 A1。

这条证据比“系统能够检索报告”更具体。它说明一个真实作者的成果进入了另一个真实使用者的后续任务，而且使用的是可定位版本。平台并非只保存文件，也不是只在验收测试里演示引用。

它的边界同样明确。运行记录证明材料被读入并形成回答，没有证明用户最终接受结论，也没有证明源报告的全部判断正确。回答中是否向用户充分展示了引用，需要与内部来源记录区分。两者都重要，但不能互相替代。

作者分析：这个案例体现的是研究协作中的信息连续性。 原作者不必在每次后续问题出现时重新解释全部材料；后来者也不必从零寻找版本。平台承担了材料识别、摘要组织和后续调用之间的连接工作。

下一步值得补充的证据，是用户能否从回答回到原报告、能否发现结论条件、发现错误后能否让纠正影响后续使用。只有把这些环节补齐，跨作者复用才能从“被调用”进一步走向“可放心采用”。这是一项明确的后续验证任务，不能通过扩大现有数字来代替。

40

历史完整编译：实际完成了什么

AbleWork 曾运行完整 AutoWiki 编译批次。任务创建于 2026 年 9 月 30 日，正式发布跨越 9 月 30 日至 10 月 1 日。核查的 152 个逻辑任务中，151 个完成并发布，1 个失败；正式快照保留 1,717 个页面记录。这是历史批次，不能计作近期窗口内的新增长。 [A1]

FIG. 13 / 机制示意

历史完整编译的工作路径



机制示意 · 箭头表示工作关系，不表示耗时或性能

候选、正式提交与发布分别有记录；历史验证不能代替当前默认运行状态。依据 A1。

成功结果能够对应候选、预检、内核正式提交和宿主发布回执。部分后续任务读取已有知识，再比较新材料。这说明完整编译不只是生成若干独立摘要，也存在跨材料的积累与更新。

具体汇总中，131 个任务记录了非空旧知识读取；59 个任务更新既有页面；快照中保留事实替代关系与多来源页面。这些证据支持“发生了更新与复用”，但不等于所有更新判断都经过专家确认，也不应把任务数当成人数。 [A1]

作者分析：完整编译最有价值的能力，是让后来的材料能够改变已有知识，并保留改变的依据。 如果每次只是追加一篇摘要，旧结论仍然存在，却没有说明是否被修订。真正的增量维护需要处理支持、更新、冲突和失效。

历史结果也暴露了工程成本：并发候选需要重新比较基线，失败要保留产物，正式提交与可读发布需要分别确认。能够跑完批次，证明这条技术路径已有实际成果；是否适合所有日常材料，还要依据用途、质量和维护成本选择。

41

为什么日常采用梗概，完整编译另行选择

AbleWork 当前社区增量默认使用分类、梗概与主题索引，完整语义编译通路没有作为日常默认开启，历史成果仍保留。[A1] 这个现状应直接写清，不能让读者误以为每一篇新报告都会进入完整知识编译。

选择较轻的处理方式，有明确的产品理由。社区里许多材料首先需要被发现、阅读和讨论。为了回答“这篇报告是否与我有关”，不一定需要抽取全部实体、关系和事实。完整编译更适合反复使用、需要更新和跨材料比较的内容。

作者分析：处理深度应跟随使用目的。 一次性动态可以保留来源与梗概；持续研究的核心对象值得建立正式知识；存在重大争议的判断可能先保留原文与候选，不急于纳入共同知识。选择深度并非给材料排尊卑，而是控制维护责任。

使用需求	适合的处理	仍需保留
判断是否值得读	分类、梗概	作者、时间、原文入口
回答局部问题	检索后精读	引用位置、条件、版本
持续跟踪对象	增量知识编译	更新关系与历史
处理分歧	并列材料与审查	争议双方、待决问题

梗概也有质量风险。已有核查材料记录过条件被压成直接预测，以及摘要增加原文没有的立场。修复需要回到具体语句和来源，不能用任务状态成功来覆盖语义问题。这个经验说明，轻处理仍然需要合适的质量检查。[A1]

因此完整编译与梗概的关系应写成分工：前者承担较重的知识维护，后者承担日常发现与导航。未来可以根据真实复用频率和错误类型调整选择，不必把一次取舍固化为永远不变的架构信条。

42

AutoWiki 与 AbleWork：语义编译和宿主责任

AutoWiki 的公开项目说明了新材料与已有知识比较、更新页面、保留来源和处理冲突的基本方法。它提供围绕知识库的取材、编译、查询与治理能力。AbleWork 将知识编译接入自身的材料、任务与发布流程。 [A4] [A1]

FIG. 14 / 机制示意

编译语义与应用边界分工



上方为编译路径，下方为宿主责任；图为职责简化，具体接线以部署为准。依据 A1、A4。

可以把两者关系理解为：AutoWiki 回答“这份材料怎样影响已有知识”；AbleWork 回答“这是谁的材料、当前允许怎样使用、何时执行、结果向谁开放”。模型生成候选，编译内核检查并提交，宿主确认版本与权限后发布。几个责任不能因同一次工具调用而混成一个。

作者分析：这种划分使知识语义可以复用，业务边界由具体应用承担。 相同的编译方法可能用于个人文件库或平台结构化存储，但每个宿主仍需提供真实身份、材料版本和写入规则。不能因为内核支持提交，就默认某个调用者有权公开全部结果。

独立内核也便于避免多套语义逐渐分叉。若每个服务自己定义“更新”“冲突”和“替代”，相同材料可能产生不同知识关系。集中维护比较与提交规则，有助于让结果一致；宿主则保留对应用流程和读权的控制。

这并不意味着所有语义判断都能由确定性代码完成。来源是否足以支持结论、两个观点是否真正矛盾，仍需要模型或人的判断。内核能够验证结构和约束，不能把这种验证夸大为全文真实。报告把可追溯性、结构正确性与语义可靠性分开评价。

43

主动准备：明确关注以后，后台继续检查

AbleWork 的运行材料记录了有边界的主动准备路径：个人建立订阅，后台周期检查事实与候选，比较当日事实水位，必要时调用模型组织卡片，经过结构与链接检查后保存，用户回来查看并继续追问。 [A2]

FIG. 15 / 机制示意

关注以后，后台准备结果



机制示意 · 箭头表示工作关系，不表示耗时或性能

同日水位未变可跳过模型和重写；产物保存与用户采用分开统计。依据 A2。

这里的主动性来自已经建立的关注范围。系统不需要等用户每天重新提出相同问题，但也不应据此自行追逐任意研究目标。周期检查是一次执行机会；是否生成新结果，仍取决于事实与已有产物。

当前机制将唤醒与模型调用分开。同日事实水位未变时，可以跳过模型和重复写入；跨日仍可能生成新一期。因此不能把这个做法扩大描述为“只要事实不变，任何时候都完全零调用”。准确的条件说明有助于后续评估成本。 [A2]

作者分析：这种方式符合研究工作的节奏。 很多变化值得提前准备，却不值得立即打断。结果进入用户回来时会看到的位置，可以降低通知压力。真正紧急或需要决定的事项，再使用明确的提醒规则。

当前证据证明已有调度和保存产物，不自动证明用户阅读或采用每张卡片。结构与链接检查也不能保证所有表达都准确。后续应把有用提醒、无效提醒、遗漏和用户纠正记录下来，才能判断主动准备是否实际节省注意力，而非只是增加后台输出。

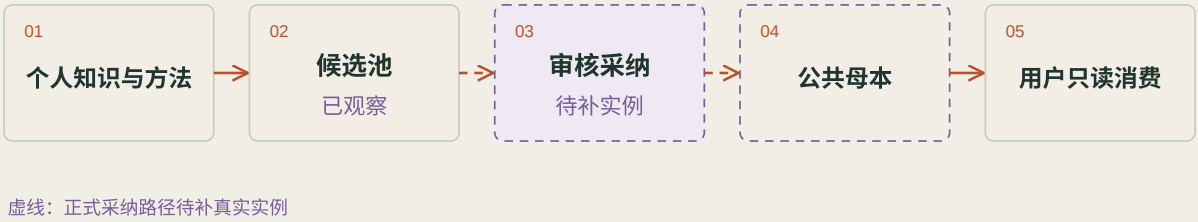
44

只读公共面与候选池：分享不等于直接改写

2026 年 11 月 1 日的生产只读核查确认，个人实例消费的 commons 公共目录以只读方式挂载，个人成果向治理候选池的汇聚已经发生；但未观察到候选正式纳入 commons 数据正典的发布回执。因此可证明的上行终点目前是候选池。 [A3]

FIG. 16 / 机制示意

个人候选到公共知识的边界



消费只读与候选上行已确认；候选到正式公共数据的采纳回执未观察到。依据 A3。

“公共层只读”需要说明读者是谁。它表示消费实例不能直接改写公共母本，不表示整个系统永远没有写入者。经过授权的发布者仍然需要维护公共内容。把消费与发布分开，可以避免个人任务无意覆盖所有人共享的知识。

候选池则保留一种低风险的贡献方式。个人 Agent 可以提出有用知识或方法，附上依据与适用范围，等待检查。提交候选并不自动赋予它正式地位，也不意味着所有私人记忆都应上行。哪些内容适合共享，需要依据类型、来源和授权判断。

作者分析：这个设计在分享便利与共同知识稳定性之间做了取舍。 代价是采纳流程可能变慢，候选也可能长期无人处理。只读边界本身不能解决治理资源不足。若候选池不断增长，却没有明确评审与发布者，就不能把它称为完整知识循环。

下一步需要真实走完少量候选的审核、正式发布、回执与消费。观察重点不是候选数量，而是读者能否使用被采纳内容、作者能否知道处理结果，以及错误能否撤回。这些仍待完成的工作应作为明确缺口呈现，而不是包装成“刻意留下的接口”。

45

报告、社区知识与 commons 不是同一块公共层

AbleWork 的不同知识路径容易因为“共享”“公共”这些词混在一起。作者报告的可见发布、社区梗概或完整知识投影、个人实例读取的 commons，各自有不同对象与写入方式。某条路径已经发布成功，不能推导另一条路径也已完成。 [A1] [A3]

路径	核心对象	当前证据	不应混入的结论
作者成果发布	报告、站点及版本	多人真实发布	代表全部知识已正式编译
社区日常导航	来源梗概与主题索引	后续问答有调用	摘要等于完整领域知识
历史完整编译	正式知识及读面	提交与发布能对应	当前每篇新材料都走此路
commons 上行	个人候选与公共母本	只读消费、候选汇聚	候选已正式采纳并复用

这种区分不是文字上的谨慎，而是影响下一步建设。社区发布需要关注作者与版本；完整编译需要关注新旧知识比较；commons 需要关注候选评审、正式采纳和回执。把它们当成一个系统状态，就无法准确定位缺口。

作者分析：平台可以有多条完整程度不同的路径，同时仍是一个已经投入使用的项目。 成熟应用往往如此。报告应说明用户今天能够完成什么、什么需要额外操作、什么还没有真实证据。项目价值不要求把全部路线都写成已完成。

对外介绍时，可以先讲最常用、证据最清楚的成果发布与后续问答路径，再介绍完整编译能力，最后说明个人到公共知识的治理方向。这样读者能够先理解实际应用，再评估扩展空间，不会被多个“层”的概念淹没。

这也修正了最初提纲中的一处风险：不能为了呈现“完整项目”，把“触发、编译、目录、边界、再消费”写成所有路径都完全相同的一条线。完整介绍应包含实际分路、异常处理和证据空白，才能支持可信的产品判断。

46

失败与反馈：把运行问题变成产品改动

AbleWork 的核查材料保留了真实使用反馈进入开发和发布的案例：一名非本人用户提出 7 条数据页面反馈，改动上线并逐条回复；同日该用户继续针对改后的页面提出建议。已有交付与再次使用证据，但没有原反馈者最终接受的完整记录。[A1]

这类证据与功能清单不同。它说明平台已经面对实际使用中的歧义，例如统计窗口、曲线选择、数值对应和导航。用户提出的问题往往不会准确对应某个模块，而是表达“这里不好理解”。产品工作需要把体验问题转成可验证改动。

作者分析：研究平台的质量需要同时检查数据表达与知识表达。 数值计算正确，但展示范围让人误读，仍然会影响判断；摘要保留了关键词，却删掉关键条件，也可能改变原意。工程成功与用户理解之间存在一段需要反复修正的距离。

反馈处理应保留原问题、采用的解释、实际改动和后续状态。若用户表达含糊，可以先做可逆的改善，并明确尚待确认的部分。不能因为代码已经部署，就把用户问题关闭成“完全解决”；也不能因为还有后续建议，就否认前一轮交付。

Agent 可以帮助归类反馈、定位相关对象、整理证据和生成修复建议，但产品裁定仍需要了解业务语义。一个看似重复的指标，可能是不同时间口径；一个看似旧版本的报告，可能用于历史回看。自动清理之前，应先弄清用户为什么需要它。

后续衡量应关注纠正是否真正影响之后的体验。某条摘要错误修复后，同类任务是否还会出现相同问题？某个页面说明改善后，用户是否仍需重复询问？这些问题比单纯统计关闭多少反馈，更能检验平台是否在变得可靠。

47

当前完整度：已运行、历史验证、待补证据

AbleWork 可以作为一个完整项目介绍，但“完整”应指定位、用户路径、工程责任和证据能够相互解释，不代表全部设想都已上线。下面按证据状态给出当前判断，截止于本报告采用的 2026 年 11 月 1 日核查资料。 [A1] [A2] [A3]

状态	已能说明的内容	仍需注意
当前运行	成果发布、梗概导航与后续问答	调用不等于答案获接受
当前运行	订阅后的周期检查与结果准备	保存不等于用户阅读采用
历史完整验证	候选、比较、校验、提交、发布、再读取	不代表当前默认全量运行
当前部分路径	commons 只读消费与候选上行	未观察到正式采纳回执
已有设计方向	更完整的对象与持续任务交互	不能仅凭设计文档认定交付
尚缺证据	全量语义质量、最终采用和效率收益	需要专门评价与用户记录

作者分析：应把有意取舍与尚未完成分开。 日常优先梗概，是根据用途选择处理深度；正式发布权集中在宿主，是边界设计；候选没有完成采纳实例，则是尚缺的路径证据。三者不能都用“暂留接口”概括。

这个区分也使规划更清楚。有意取舍可以长期保留，并通过使用反馈调整；证据缺口需要安排实际案例验证；工程缺口需要实现和验收。不同问题对应不同动作，避免用继续讨论定位来拖延具体交付。

对读者最诚实的表述是：AbleWork 已让多位使用者发布并复用研究成果，也曾跑通完整知识编译；正在把这些能力组织成更稳定的持续研究体验。下一阶段的重点，是补齐正式采纳、纠错传播和用户采用证据，而不是先把能力清单写得更长。

48

AbleWork 的结合路线：先增强已有工作

结合其他形态，应从 AbleWork 已有用户路径出发。最值得优先增强的，是研究员已经在做、平台已经有对象和证据的工作。每个借鉴都应能够说明它减少了哪项负担，以及怎样验证没有引入新的边界问题。

借鉴方向	可落在 AbleWork 的动作	验证条件
Muse 的持续关系	保存关注原因、期限和用户修正	少重复解释，旧关注可停用
Grok Bot 的流程复用	固化高频研究方法与例行检查	异常样本可处理，方法有版本
Claude Tag 的原处协作	委托、进展与成果链接到对象	同事能接续，读权保持正确
Super-agent 的任务状态	显式表达等待、恢复与交付	中断后不丢成果、不重复行动
知识治理的深化	候选采纳与纠错影响后续消费	回执、版本与撤回可核对

近期可以选一项持续主题作为贯穿案例：用户建立关注，后台检查变化，Agent 准备有出处的更新，用户确认其中有长期价值的内容，再让另一位有权读者用于后续问题。每一步都保留输入、状态和结果，而不要求同时扩展所有领域。

作者建议：先补齐已发生复用的解释与纠错，再扩大自动化范围。 如果用户看不见来源，更多自动摘要只会增加难以判断的内容；如果候选无人采纳，扩大候选生成量不会形成公共知识；如果等待状态不清楚，新增更多主动任务会增加追踪负担。

这种建设顺序不排斥更开放的研究 Agent。它为开放任务提供必要基础：知道自己在依据什么工作，哪些条件改变了，谁可以决定下一步，以及结果如何进入共同知识。后续形态越自主，这些基础越重要。

AbleWork 的方向因此可以保持聚焦：让研究工作有连续对象，让成果有可靠归属，让复用有边界。具体执行器、模型与交互入口可以变化，用户已经积累的知识与工作记录应能够继续使用。

49

情境推演：一份持续研究委托怎样分工

设想一位研究员提出：“持续关注一个行业的供需变化，每周准备更新，有重要证据时修订研究框架。”这是情境推演，不对应任何产品的实测成绩。它同时包含长期目标、例行流程、资料更新和知识维护，适合检查几种形态如何组合。

个人代理可以先帮助明确关注目的：哪些问题尚未回答，研究期限多长，哪些变化值得提醒。角色 Bot 可以承担稳定的资料整理方法。团队频道可以讨论新证据，确认是否改变原有假设。任务系统记录等待的数据与审查。知识平台保存正式成果和更新依据。

FIG. 17 / 机制示意

持续研究委托中的责任交接



机制示意 · 箭头表示工作关系，不表示耗时或性能

情境推演：口径分歧与重要结论变化需要明确裁定，不能只按时间覆盖。

真正困难的节点出现在分歧。一份新材料支持旧判断，另一份材料却使用不同统计口径。系统不应简单按发布时间选较新结论，也不应把两者平均成一段模糊表述。它需要把口径差异与待决问题交给合适的人，再记录采用哪种解释。

用户可能只希望每周看到摘要，却在重要变化时愿意即时介入。这要求通知规则与处理方法分开。系统可以每天检查，但只有满足既定条件时提醒；也可以在证据不足时先积累候选，不频繁制造“重大变化”。

对 AbleWork 的验证重点，是整个过程能否围绕同一个研究对象接续。新资料、讨论、任务和最终版本应能相互找到。若每次更新都生成一篇与过去无关的新报告，虽然内容增加了，持续研究的责任仍然落在人身上。

50

情境推演：多人接力修复一个问题

设想团队发现数据页面上的某个读数含义不清，请 Agent 调查。第一位成员只看到异常，第二位成员了解业务口径，第三位成员负责工程实现。这个情境能够检验团队协作与任务接续，也说明为什么单个高质量回答还不够。

频道形态适合保留共同讨论：异常何时出现，谁提供了新线索，目前采用什么解释。持续任务形态适合记录执行状态：是否已经复现、是否等待数据口径确认、修改是否可供审查。研究平台则应把修正与受影响的指标、报告或主题关联起来。

作者分析：任务交接应保留已经排除的解释。 如果后来者只看到最终一句“请修复显示”，可能重新走一遍排查，或者把有意保留的历史口径删掉。失败尝试与被排除假设并非都需要长篇展示，但应在需要时能够查回。

这里的授权也不是一次性笼统批准。调查和制作可审查改动，可以在已委托范围内继续；生产发布、改变数据定义或删除历史记录，可能需要不同责任人确认。系统应该把需要决定的具体差异呈现出来，让用户批准一个可检查结果。

验收应同时覆盖技术与阅读。测试可以证明计算或接口行为符合预期，真实页面检查则能发现标题、单位、范围和提示是否使用户理解错误。若只看测试通过，用户原本的困惑可能仍然存在。

AbleWork 已有真实反馈推动页面改动的案例，但本页是更一般的推演。后续可以把它形成可复用方法：保留原问题，列出采用解释，生成可审查结果，确认发布，再记录用户是否继续遇到同类问题。这样 Agent 协作围绕用户困难展开，而不是围绕内部步骤自我完成。

51

情境推演：来源更新或撤回之后怎么办

设想一份共享报告曾支持某个知识条目，后来作者更新了结论，或收回了共享权限。系统中可能仍有旧摘要、已生成回答、个人笔记和后台任务。这个情境检验的是知识生命周期，而非首次生成能力。

FIG. 18 / 机制示意

来源变化应影响派生结果



机制示意 · 箭头表示工作关系，不表示耗时或性能

情境推演与建议；不表示所有影响传播均已实现。历史合法交付按具体政策处理。

来源更新时，应先区分内容变化与可见性变化。内容变化可能需要重新摘要、比较知识和提示受影响任务；可见性变化则要求后续读取遵守当前权限。已经合法交付的历史结果如何处理，需要具体产品政策，不能笼统承诺可以从所有人的记忆中删除。

作者建议：至少建立可追踪的派生关系。 知道哪些摘要和知识引用了哪一版来源，才能安排更新。若只有无来源文本，系统连影响范围都无法判断。派生关系不要求每个字都有复杂标签，但重要结论应能追到依据。

后台任务恢复时也应重新核对材料条件。任务在昨天获得读权，不代表今天仍可照常读取。若来源不可用，系统应该明确说明缺口，保留已完成的其他工作，而不是悄悄用旧缓存补齐后宣称最新。

对 AbleWork，这类验证比单纯扩大知识规模更能支撑“边界治理”定位。可以选择一份测试材料，完成发布、摘要、后续调用，再更新或撤回，检查每个读面和恢复任务。只有真实跑过变化后的路径，才能说明边界在持续工作中有效，而不只是首次接入时有效。

52

情境推演：个人发现怎样成为公共知识

设想研究员在私人工作区发现一条可复用方法，希望分享给同事。个人 Agent 可以整理方法说明、适用条件和来源，再形成候选。公共知识的发布者需要检查它是否确有共享价值、是否含有私人材料，以及是否与已有方法冲突。

这里最容易出现的误判，是把“作者愿意提交”当作“所有输入都允许公开”。方法可能引用第三方受限资料，也可能包含只有特定团队才能理解的业务背景。提交候选时应明确允许分享的内容，而非直接复制整个私人研究过程。

阶段	应形成的记录	决策重点
提出候选	方法、依据、适用范围	是否值得他人复用
比较检查	与现有知识的关系	重复、更新还是分歧
正式采纳	版本、发布者、可见范围	谁承担维护责任
后续消费	使用入口与来源	读者能否正确理解
修订或撤回	原因与替代版本	旧内容如何停止误导

作者分析：公共知识需要维护者，而不只是贡献者。 大量候选可以由自动化产生，但正式知识需要持续处理过时、冲突和错误。如果没有维护能力，采纳门槛应更谨慎，先共享原始成果与候选状态，避免制造无人负责的“标准答案”。

AbleWork 当前 commons 路径已确认只读消费与候选汇聚，正式采纳实例仍需补齐。本情境可直接用作下一项端到端验收：选择少量真实、可公开的方法，完成从候选到发布再到他人使用的全过程，并保留回执与纠错方式。[A3]

53

未来判断一：入口会更自然，责任对象会更明确

从五个案例可以提出一个有条件的判断：用户入口可能继续趋向自然语言与就地委托，底层工作对象却会更明确。原因很直接，用户不愿学习复杂配置，但持续系统必须记住目标、范围、状态和结果。自然交互与结构化责任并不矛盾。

这个判断并不意味着聊天将替代所有界面。审批需要展示具体对象与后果，计划需要显示依赖，研究报告需要完整阅读，知识更新需要比较新旧。不同内容需要合适载体。自然语言适合表达意图，稳定界面适合检查和操作明确对象。

作者推断：未来 Agent 产品的重要设计能力，是在对话与对象之间转换。 用户说“继续关注”，系统应让关注对象、期限和停止方式变得可见；用户说“采用这个结论”，系统应知道采用的是哪一版、适用于什么范围。无法完成这种转换，长期任务就容易停留在含糊承诺中。

反例也存在。短暂问答、创意探索或低后果个人工作，未必需要正式任务对象。强行结构化会增加负担。因此产品应按工作长度、影响范围和复用需求逐步增加结构，而不是把每一句话都变成管理记录。

验证这一判断，可以观察用户是否愿意在关键节点确认对象，而不需要理解内部技术。例如看一张清楚的“持续关注”卡片，就能知道系统接下来会做什么。若为了完成简单委托必须填写大量字段，说明结构暴露过多；若用户无法解释系统承诺了什么，则结构不足。

AbleWork 的机会在于已有研究对象。报告、主题和指标天然提供上下文与稳定入口，可以减少用户描述成本。下一步应让任务与成果版本更容易回看，同时保持用户围绕研究问题工作，而非围绕系统内部流程工作。

54

未来判断二：Agent 身份会成为应用设计的一部分

当 Agent 只给建议时，身份问题不太显眼；当它读取多个账户、修改系统并代表团队发言时，身份就直接影响责任。前文产品已经以不同方式处理个人连接、服务账号、授权任务和发布主体。未来应用很可能需要把这些关系作为常规产品对象。

作者推断：组织将更重视“这个 Agent 以谁的身份做了什么”，而不是只问它用了什么模型。相同模型在不同权限下具有完全不同的实际能力。一个可以读取共享文档的 Agent，与一个可以修改生产配置的 Agent，不应只靠角色名称区分。

这会影响采购、审计和日常体验。管理员需要知道哪些连接仍在使用，用户需要知道任务会以什么身份行动，结果接收者需要知道来源与责任。身份记录应服务于这些具体问题，避免成为只有管理员看得懂的庞大权限表。

角色与身份可能形成多对多关系。同一个角色可以在私人任务中使用个人权限，在团队任务中使用服务账号；同一个任务也可能经过多个执行者。系统应保留任务授权，而不是让执行者替换时丢失约束。角色是交流方式，身份是行动依据。

这一判断成立的条件，是 Agent 持续扩大真实工具使用。如果大量应用仍停留在低后果内容生成，复杂身份设计可能没有足够收益。因此建设者应从实际写入与共享场景开始，先解决最容易出错的归属，再逐步扩展。

AbleWork 的优势不是宣称比别人更安全，而是可以在明确研究场景中把作者、执行者、发布者和读者分开。通过真实跨作者复用和撤回案例验证这些关系，比增加抽象治理口号更能说明平台价值。

55

未来判断三：可携带成果比无限对话更耐久

长期 Agent 会积累大量对话，但用户真正需要带走的，通常是目标、成果、方法和可靠知识。完整聊天历史可以帮助追查，却不一定适合直接作为下一次工作的主要输入。随着任务变多，选择和组织上下文会比单纯保存全部历史更重要。

FIG. 19 / 机制示意

让成果离开一次对话仍可使用



机制示意 · 箭头表示工作关系，不表示耗时或性能

可携带对象不等于公开对象；迁移与复用仍受授权约束。作者推断。

作者推断：成果的稳定身份、来源与版本，会逐渐成为跨 Agent 协作的重要接口。 不同执行者能够围绕同一份研究结果继续工作，用户就不必把全部经验绑定在某个对话或角色中。这里说的是发展方向，并非已有统一行业标准。

可携带不等于无条件公开。一个成果可以在用户自己的工具之间迁移，仍然保留私人范围；团队成果可以被授权系统读取，而不必暴露底层凭据。交换的对象应包含用途与边界，不能只把正文复制出去。

知识编译在这种形态中有明确作用：把重复使用的内容整理成可更新对象，让新证据能够改变旧结论。检索仍然重要，它帮助发现相关材料；编译帮助维护被反复依赖的知识。两者可以配合，不需要用互相否定来证明价值。

AbleWork 可以把这一方向落实到具体交付：稳定的报告入口、可核对版本、来源与更新关系，以及对不同执行者清楚的读写接口。只要这些对象保持可用，未来更换模型、执行框架或交互界面，就不必重新建立全部研究积累。

56

未来判断四：多个形态会并存，运营能力成为分水岭

五种形态不太可能简单排成一条从低到高的替代路线。个人事务、团队讨论、工程任务和领域研究具有不同工作节奏与责任关系。能力会相互吸收，但产品仍可能保留不同入口和组织中心。一个适合家庭事务的代理，不必同时成为组织知识的正式发布者。

作者推断：产品差异将更多体现为长期运营能力。 连接失效后怎样处理，任务卡住后怎样恢复，资料更新后怎样纠正，用户离开后怎样停止，预算不足时怎样降级。这些问题在演示中不醒目，却决定系统能否持续被使用。

这种能力可以由平台、执行框架和组织流程共同承担。应用不必自己实现每个组件，但必须对用户交付整体结果。若一个任务跨越多个服务，用户不应被迫自己判断究竟是哪一层失败，至少应该获得清楚的当前状态与可行下一步。

反例是高度标准化、短周期工作。如果输入输出固定、错误容易发现，轻量自动化可能长期足够。建设持续 Agent 应有真实需求依据，避免把简单规则系统能做的工作包装成复杂协作。更强的模型不意味着每个流程都需要更多自主性。

评价未来形态时，可持续观察几个信号：用户是否反复回来，是否愿意扩大委托范围，错误后是否还能继续使用，以及成果是否被不同人和不同任务采用。它们比一次华丽演示更慢，但更接近产品实际价值。

AbleWork 的下一阶段不需要证明自己会取代所有 Agent。只要能让研究成果在权限范围内持续进入后续工作，并降低维护这些关系的成本，就有清楚的应用位置。其他执行产品越强，可靠成果托管与知识更新的需求也可能越明确；这一推断仍需真实采用数据检验。

57

怎样做一轮公平、可复核的比较

如果要把本报告进一步用于产品选择，应补做同任务验证。先选择各候选都适用的一小组真实工作，固定输入版本、允许工具、预算和验收条件。超出某产品范围的任务应单列，不宜直接记为能力差。比较的对象是配置后的完整工作方式。

检查项	应记录的证据	不应替代它的指标
结果可用性	盲审或业务验收及理由	输出长度、语气自信
接续能力	中断、恢复、追加要求的记录	单次运行时长
权限正确性	越界尝试、撤回与拒绝路径	提示词写了“安全”
人工负担	准备、纠正、审批与追踪	模型耗时单项
复用质量	后续任务引用与更新正确性	存储页数、召回次数
总成本	资源、人工与失败恢复	单次调用价格

任务集应包含正常案例和变化案例。来源缺失、输入更新、授权撤回、执行中断、相互矛盾的要求，都应在可控环境中出现。否则测试容易只证明系统沿着预先准备的顺利路径运行。

作者建议：让使用者评价结果，同时让独立检查者核对过程。 使用者能判断是否有帮助，过程记录能说明结果怎样形成。两者出现分歧时应保留：用户满意不代表来源正确，检查全部通过也不代表解决了真实问题。

报告没有执行这轮跨产品实测，因此不提供总分或排名。这里给出的是下一步方法。对于AbleWork，可以先选已经有运行证据的跨作者复用任务，再补上用户采用、来源更新和撤回路径，形成更强的应用证明。

58

建设路线：从一个可靠案例，走向持续职责

建设者可以按证据逐步扩大责任，而非按功能清单推进。第一步选择一个高频问题，明确用户最终要拿到什么。第二步把一次工作做成可检查结果。第三步验证中断、纠正与权限变化。第四步才考虑例行化和多角色协作。每一步都应留下真实案例。

FIG. 20 / 机制示意

按证据扩大责任



机制示意 · 箭头表示工作关系，不表示耗时或性能

建议建设顺序，无日期承诺；每一步以真实结果和异常路径验证后再扩大范围。

对 AbleWork，优先工作可以分为四项。补足回答到原文的可见追溯；选择少量候选完成正式采纳与回执；验证来源更新或撤回后的影响；为真正跨天的研究任务提供清楚等待状态。这些动作都能增强已有应用，不必等待一个全新的 Agent 架构。

验收条件应具体到用户行为。读者能否找到正确版本，作者能否知道候选是否采用，任务中断后能否继续而不重复发布，权限变化后是否停止不再允许的读取。每项都可以通过有限样本得到明确结果。

作者建议：自动化范围随着证据扩大。 一个流程在正常样本中有效，只足以进入有限试用；经历不同输入、异常和纠正以后，才适合承担更稳定职责。失败记录应参与下一次方法修订，不能仅被清理成成功状态。

研究平台最终需要守住的，是工作与知识之间的责任链：有人提出问题，系统在允许范围内推进，成果由明确主体确认，后来者知道依据和版本。模型、角色和入口可以不断变化，这条责任链应能持续解释系统正在做什么。

59

公开来源：外部产品与研究原型

以下为本报告使用的主要一手资料，读取截止 2026 年 11 月 1 日。页面可能继续更新，正文描述对应本次读取范围。产品机制采用官方说明，比较与建议由作者提出；未将官方介绍视作独立性能测试。

M1 Meta · Introducing Muse
2026-09；个人代理定位、执行环境、权限与产品计划。

M2 Meta · How We Designed Muse
2026-09；目标、持续对话、活动记录与成果体验。

M3 Meta · How We Built Safety Into Muse
2026-09-08；执行隔离、Sentinel、凭据与计划中的 Confidential VM。

G1 Grok Bot · Overview
页面标注更新 2026-10-28；角色、共享电脑、协作与 X 入口。

G2 Grok Bot · Skills and routines
方法复用、示范、例行触发与边界。

G3 Grok Bot · Approvals, security, and privacy
行动前检查、用户审批与外部内容的区别。

C1 Anthropic · Introducing Claude Tag
2026-06-23；团队协作、异步交付与可选主动行为。

C2 Claude Tag · Overview
频道使用、临时执行环境与原线程交付。

C3 Claude Tag · How agent identity works
服务账号、频道范围、个人连接器与凭据代理。

C4 Claude Tag · What Claude Tag remembers
共享上下文及记忆范围；与身份文档结合阅读。

D1 DeepSeek · dsh-superagent-prototype README
公开研究原型；非官方生产产品，与内部版本及文章范围有别。

D2 Super-agent · Domain model / Authorization · Authorization
事件、任务、授权、派工及上下文订阅。

D3 Super-agent · Execution
状态、执行目录、等待、失败与恢复。

D4 Super-agent · Philosophy
注意力、规则授权与发言原则。

D5 Super-agent · Deployment
部署前提与执行隔离边界。

来源使用原则：同一产品的介绍页用于确认定位，使用文档用于确认工作方式，安全或身份文档用于确认特定边界。研究原型以其明确公开的实现范围为准，不把设计愿景自动写成已交付功能。本文未复制产品界面；所有配图均为作者绘制的机制示意。

60

AbleWork 依据、统计口径与研究限制

A0 AbleWork · 公开项目介绍
产品定位、两条知识处理路径与持续委托。

A1 作者运行核查 · AbleWork 实际运行与产品成果
2026-10-31 23:37:37 CST 截止；本文使用脱敏汇总，底稿不公开。近期使用与历史完整批次分别统计。

A2 作者运行补核 · Able 主动准备路径
2026-11-01 00:17:35 CST 截止；调度、事实水位、卡片保存与现役代码的只读核查，底稿不公开。

A3 作者运行核查 · commons 读面与候选上行
2026-11-01 21:05—21:25 CST；只读挂载、召回、候选汇聚与正式采纳回执检查，底稿不公开。

A4 AutoWiki · 公开项目与方法说明
知识比较、编译、查询与治理；具体宿主版本及运行状态另据 A1。

作者运行核查为自有项目的一手观察，非第三方独立审计。公开汇总省略内部业务材料、用户标识、服务地址与原始日志。读者可核对公开项目定位；内部运行数字只能按本文披露的口径理解，不能视为已公开可复算的数据集。

近期窗口为北京时间 2026 年 10 月 1 日 23:37:37 至 10 月 31 日 23:37:37。使用者按可识别个人主体去重，代人 Agent 不另计用户，排除测试与后台活动。成果按作者与成果标识去重，多版本不重复计数。来源调用证明进入任务输入，不证明答案正确或用户采纳。

历史完整批次任务创建于 9 月 30 日，正式发布跨至 10 月 1 日；152 单中 151 单发布、1 单失败，1,717 页为正式快照记录。它与近期梗概任务属于不同集合，不能相加作为月度产出。commons 核查截止 11 月 1 日晚，候选正式采纳仍缺实例证据。

本报告没有测量跨产品准确率、效率提升、投入产出比或安全强弱。后续若补充实测，应保留任务、输入版本、工具范围、预算、人工干预和验收记录，再修订相应判断。报告编成于 2026 年 11 月 2 日，所属研究时间线为 2026 年 11 月。